Joule Studio

Generated on: 2026-08-14 17:42:54 GMT+0000

Joule Studio | Cloud

Public

Original content: https://help.sap.com/docs/JOULESTUDIO/uacp_sap-business-ai-platform-docs_joule-studio?locale=en-US&state=PRODUCTION&version=CLOUD

Warning

This document has been generated from SAP Help Portal and is an incomplete version of the official SAP product documentation. The information included in custom documentation may not reflect the arrangement of topics in SAP Help Portal, and may be missing important aspects and/or correlations to other topics. For this reason, it is not for production use.

For more information, please visit https://help.sap.com/docs/disclaimer.

What Is Joule Studio?

Joule Studio is SAP's AI-first environment to create enterprise solutions from intent to deployment. It is SAP's AI coding tool with low-code capabilities to create and run agents and other solution types with the governance and business understanding required to deliver production-ready agentic solutions on the SAP Business AI Platform.

Joule Studio follows the intent-based development approach. Users start by describing the outcome they want, and Joule Studio helps translate that intent into working solutions.

Access

You access Joule Studio via Joule Work. Just select ***Develop*** in the left navigation. Alternatively, you can also state your intent in the ***Conversations*** area of Joule Work. If you are entitled to use Joule Studio, this will start the intent-based development process.

You can also access and use the features of Joule Studio through the `jl` CLI.

Joule Work

Joule Work is the overarching intelligent workspace UI that adapts to how you work, surfacing relevant insights, and coordinating AI assistance across your organization. Joule Studio is the environment to develop new AI-based solutions. Joule Studio is integrated with Joule Work and can be accessed via the ***Develop*** area in the left navigation. If you need more information about Joule Work and its capabilities, check the Joule Work documentation.

Getting Started

To learn how to build your first solution, see Getting Started: Create an Agent in Joule Studio.

Two Ways to Build

Joule Studio offers two paths to build solutions, both following the same intent-based development workflow.

Path	Audience	Surface	Tutorial
UI	Business users, citizen developers	Browser-based Joule Studio in Joule Work	Getting Started: Create an Agent in Joule Studio
Pro-Code	Professional developers	`jl` CLI in the terminal, optionally with an AI coding agent	Getting Started: Create an Agent with the CLI

Both paths build the same kinds of solutions and deploy to the same managed runtime service. For an introduction to pro-code development, see Pro-Code Development in Joule Studio.

Key Advantages of Joule Studio

- Describe your business challenge and solution ideas in natural language.

- Receive end-to-end guidance from Joule, from idea to deployment.
- AI-driven Discovery step designs an SAP-optimized solution.
- Significantly accelerated development time.
- Accessible to business users and citizen developers without local tooling.

Joule Studio Solution Types

Joule Studio generates agents that connect to your SAP systems, answer questions, and take action on your behalf.

Solution Type	Description	Business Value	Example
Agents	AI-powered assistants that understand natural language, access SAP data, and execute tasks. They answer questions, perform operations, and deliver insights through conversation, autonomously completing multi-step processes that would otherwise require manual effort.	Replace manual, time-consuming processes with intelligent automation grounded in your SAP data	Missed Discount Analysis Agent: ask "how much am I losing on late payments?" and it calculates the impact from SAP S/4HANA data

For more information, see Solution Types.

Intent-Based Grounding Framework

Joule Studio's intent-based development approach is a grounding framework that anchors every solution in your actual business intention. The six steps guide you from idea to deployment, but they are not a rigid sequence. You can revisit earlier steps, refine your intent after seeing the generated specification, or loop back after testing to adjust requirements.

1. Intent

After you provide your intent, Joule Studio performs AI-driven discovery. It then asks clarifying questions to validate the analysis.

2. Requirements

The AI transforms the intent into a formal Product Requirements Document (PRD) with user personas, requirements as user stories, scope definition, and high-level architecture.

3. Specification

The AI generates architecture decisions, API references, data models, and a prioritized task list. It discovers relevant SAP APIs from Business Accelerator Hub.

4. Solution

The AI generates production-ready agents, automations, and applications tailored to your SAP landscape.

5. Testing

The solution deploys to an isolated sandbox for validation.

6. Deployment

The AI packages and deploys to a managed runtime solution, including monitoring dashboards.

For more information, see The General Development Flow.

AI-Driven Solution Discovery

Joule Studio proposes intelligent solutions based on a four-part solution discovery process. During this process, Joule Studio explores the following tools:

Discovery Step	Purpose	Benefit
SAP LeanIX	Enterprise Landscape Understanding: Provides information about your specific SAP environment, including deployed systems (SAP S/4HANA, SuccessFactors, Ariba), deployment models (Cloud/On-Premise/Hybrid), existing integrations, and organizational structure.	Ensures that generated solutions work within the actual landscape.
SAP Knowledge Graph	API Discovery and Architecture Mapping: Maps business needs to SAP's Reference Business Architecture, identifies relevant APIs with specifications and authentication requirements, and performs fit-gap analysis to determine what can be reused versus what needs custom development.	Ensures solutions align with SAP best practices and developers don't need to manually search the SAP Business Accelerator Hub.
SAP Domain Models	Best Practices and Intelligence: SAP-opinionated AI models guide solution architecture decisions, design patterns, security/compliance guidance, and performance optimization strategies.	The generated code follows proven SAP patterns, not generic approaches.
Fit-Gap Analysis	Compares what already exists in standard SAP offerings (fit) vs. what needs custom development (gap). It identifies reuse opportunities and gaps requiring custom code.	Allows you to leverage existing SAP capabilities where possible.

For more information, see Intent Refinement with AI-Driven Discovery.

SAP-Managed Provisioning for Joule Studio

ℹ Note

Important: Please check the information on SAP-managed provisioning at SAP-Managed Provisioning in the Joule Work documentation.

Before anyone in your organization can use Joule Studio to build agents or other solutions, a systems administrator must provision three components in SAP for Me, in the following order:

1. **Core components**: Core components provide foundational services that Joule relies on. For more information, see Provision Core Components.
2. **Joule**: You set up Joule and configure integrations with your SAP systems. For more information, see Provision Joule.
3. **Joule Studio**: You provision Joule Studio to build custom AI agents, workflows, and applications. For more information, see Provision Joule Studio.

ℹ Note

Important: All provisioning and role assignment steps must be performed separately for each environment. Do not skip the non-production environment. It's required before you can set up the productive environment.

Each provisioning wizard follows the same four-step structure:

Step	What you do
1. General Information	Give the solution a name and confirm the contract details.
2. Set Up Systems	Select the SAP systems that the new solution should integrate with.
3. Set Up Formations	Confirm or rename the formations that will be created.
4. Review	Verify your selections and start provisioning.

Prerequisites

- Your organization has the following commercial entitlements in place:
  - You have a valid license for an SAP product that supports Joule integration in one of the supported regions.
  - You have purchased the AI Unit SKU (8019164) or Joule Base.
  - You have an active BTP Enterprise Agreement (BTPEA), Cloud Platform Enterprise Agreement (CPEA), or Pay As You Go (PAYG) contract
  - You have purchased the Joule Studio SKU (8021813). The SKU is linked to your BTPEA, CPEA, or PAYG contract.
- You have administrator access to SAP for Me.

ℹ Note

Note for existing Joule customers: You can follow this SAP-managed provisioning approach for Joule alongside an existing customer-managed Joule setup. However, applications support only one embedded Joule tenant at a time. For a consistent

user experience, SAP recommends switching fully to managed Joule rather than running both in parallel.

Procedure

Follow the three steps in order. If Core Components and Joule are already provisioned in your landscape, you can skip directly to step 3: Provision Joule Studio.

1. Provision Core Components
2. Provision Joule
3. Provision Joule Studio

Provision Joule Studio

Prerequisites

You have provisioned the core components and Joule as described in the Joule Work documentation.

Procedure

1. Log in to SAP for Me.
2. In **My Product Packages**, search for **Joule** and select the **JouleWorkDev** tile.
3. On the **Applications** tab, locate **JouleStudio Dev** and select **Start Provisioning**.
4. On the **General Information** step, enter the required information and select **Next Step**:
    - **Name**: For example, `<your-org>-joule-studio`
    - **Path**: Confirm the suggested path.
    - **Business Type**: Select either **Test** or **Production**.
5. On the **Set Up Systems** step, configure the integrations Joule Studio needs:
    - **Joule Core Dev System**: Select the Joule core dev system.
    - **SAP LeanIX System**: Select your LeanIX tenant.
6. On the **Set Up Formations** step, review the pre-filled **Formation Name**. After initial provisioning is complete, you cannot change the formation name.
7. On the **Review** step, verify your selections and select **Finish**.

ℹ Note

Note: Provisioning runs in the background. The **Creating Solution** dialog confirms that the request has been sent. You can close the dialog and monitor progress on the **Resources** tab. SAP for Me notifies you once provisioning is complete.

Verification

To verify that all three components are provisioned:

1. In SAP for Me, go to **Portfolio & Products** > **My Product Portfolio**.
2. Confirm that all three solutions (Core Components, Joule, Joule Studio) appear with status **Ready**.

Next Steps

Once provisioning is complete in both environments, assign the appropriate roles to your users. See Role Management.

Role Management for Joule Studio

Overview

Access to Joule Studio is controlled through two roles. A systems administrator assigns these roles to users in SAP Cloud Identity Services (SCI).

Each environment (non-productive and productive) has its own SCI tenant. Role assignments must be made separately in each tenant:

- Assignments in the **non-productive** SCI tenant grant access to the **non-productive** Joule Studio.
- Assignments in the **productive** SCI tenant grant access to the **productive** Joule Studio.

Roles

Joule Studio Developer

Users with this role can:

- Create, edit, and deploy solutions they have built
- Monitor their own solutions

Developers can only see and work with solutions they have created themselves. They do not have visibility into solutions built by other users.

> **Note**
>
> **Note:** Building and developing solutions is only possible in the non-productive (development) environment. Developers cannot create or edit solutions in the productive environment. Because developers only see what they have built, they will not see any solutions in the productive environment.

Joule Studio Administrator

Users with this role can:

- View all solutions across the tenant, in both environments
- Deploy and monitor all solutions

Administrators cannot build or develop solutions. That capability is reserved for users with the Developer role.

> **Note**
>
> **Note:** A future release will introduce sharing and collaboration features. Once available, administrators will be able to share solutions with developers, who can then monitor those shared solutions.

Procedure

Repeat these steps once for the non-productive SCI tenant and once for the productive SCI tenant.

Prerequisites

You need the **Manage Users** and **Manage Groups** administrator roles in the SCI tenant. For details, see the SCI documentation on editing administrator authorizations.

The `JouleStudioAdministrator` and `JouleStudioDeveloper` groups must exist in the tenant.

Step 1: Open the User Management Tile

1. Sign in to the SCI administration console for the relevant environment. The URL has the form `https://<your-sci-tenant>.accounts.ondemand.com/admin`.
2. On the home page, choose the **User Management** tile.

Step 2: Assign the Joule Studio Groups to Each User

For each user who needs access to Joule Studio:

1. In the user list, select the user.
2. Open the **Groups** tab.
3. Select **Assign** at the top of the page.
4. In the dialog, select the groups appropriate for the user:
   - `JouleStudioDeveloper` for users who should create, edit, deploy, and monitor their own solutions
   - `JouleStudioAdministrator` for users who should view, deploy, and monitor all solutions in the tenant

   A user can be assigned both groups if they need both roles.

   > **Note**
   >
   > **Note:** The exact group names may vary by tenant configuration. If you do not find them, search for groups containing "JouleStudio" combined with "Developer" or "Administrator".

5. Select **Assign** in the dialog to confirm.

Step 3: Repeat for the Other Environment

Repeat Steps 1 and 2 in the SCI tenant for the other environment, so that:

- Users who should develop solutions have the `JouleStudioDeveloper` group assigned in the non-productive SCI tenant.
- Users who should monitor and deploy solutions in production have the `JouleStudioAdministrator` group assigned in the productive SCI tenant.

Verification

To verify role assignments:

1. Have a user in the Developer group log in to Joule Work and select **Develop** to open Joule Studio. The **Solutions** pane should be visible and allow them to create new solutions.
2. Have a user in the Administrator group log in to Joule Studio. They should see all solutions in the tenant, but the option to create a new solution is unavailable.

Next Steps

Once roles are assigned, developers can log in to Joule Studio and start building solutions. See Get Started with Joule Studio.

Related Information

The roles on this page control who can **build and manage** solutions in Joule Studio. They are separate from agent access policies, which control which end users can **invoke** a specific deployed agent through Joule. For details, refer to Assign Agent Access.

GitHub Integration Overview

Joule Studio uses GitHub for two purposes:

- **Version control**: Developers can connect a solution to a GitHub repository and push or pull solution files directly from Joule Studio.
- **Production deployment**: Promoting a solution to the productive environment uses a GitHub-based approval gate. A developer pushes the solution to a GitHub repository, and an administrator manually triggers a GitHub Actions workflow to deploy it.

GitHub integration is not required for building, testing, or deploying solutions in the development environment.

Setup Guides

Choose the guide that applies to your GitHub setup:

Setup	When to use
GitHub Setup	Required for all organizations. Covers the one-time SCI trust configuration, GitHub repository setup, and GitHub Actions workflow needed for production deployment. Also covers how to connect a solution to GitHub from within Joule Studio.
GitHub Enterprise Setup	Required only for organizations using GitHub Enterprise. Covers the additional SAP for Me configuration needed to register your GitHub Enterprise instance as a system. Complete this before GitHub Setup.

GitHub Setup for Joule Studio

Overview

Production deployment in Joule Studio uses GitHub as the promotion mechanism. A developer pushes their solution to a GitHub repository, and an administrator manually triggers a GitHub Action to deploy it to the productive environment. This manual trigger is the deliberate approval gate for production.

> **Note**
> **When to do this setup:** You only need GitHub setup once you're ready to promote a solution to production. It's not a prerequisite for building, testing, or running solutions in the development environment. If you're just getting started with Joule Studio, skip this page and come back to it when your first solution is ready for production.

Development deployments do not require GitHub. You can deploy to the development environment directly from Joule Studio. See Deployment.

Once you're ready to promote to production, a systems administrator completes this setup once per solution before the first production deployment.

Prerequisites

- Core Components, Joule, and Joule Studio are provisioned for both environments. See Provisioning.
- Your organization has a GitHub license (GitHub.com or an enterprise license). The GitHub instance must be internet-accessible and not behind a VPN.
- You have administrator access to your SAP Cloud Identity Services (SCI) tenant.

Step 1: Establish Trust Between SAP Cloud Identity Services and GitHub

Authentication between GitHub Actions and Joule Studio uses GitHub's OpenID Connect support. When a workflow runs, GitHub issues a short-lived JWT. The workflow exchanges that JWT for a Cloud Identity Services token, which it then uses to call the Joule Studio solution management API.

To enable this exchange, configure your SCI tenant as follows.

Define an Application Representing Your GitHub Organization

1. Log in to the SCI administration console. The URL has the form `https://<your-sci-tenant>.accounts.ondemand.com/admin`.
2. Go to **Applications & Resources** and select the **Applications** tile.
3. Select **Create** and fill in the dialog:
    - **Display Name**: Use a name that identifies your GitHub organization, for example `github.com/<your-org>`.

- **Type**: Non-SAP solution
- **Parent application**: None
- **Organization ID**: global
- **Protocol Type**: OpenID Connect

4. Select **Create** and select the newly created application.
5. On the **Trust** tab, under **Application APIs**, select **Client Authentication**.
6. Fill in the dialog:
    - **Description**: Optional description
    - **Issuer Configuration**: Select **Manual Configuration**.
    - **Issuer**: Enter `https://token.actions.githubusercontent.com`.
    - **JSON Web Key Set URI**: Keep the default value `https://token.actions.githubusercontent.com/.well-known/jwks`.
    - **Refresh Interval**: Select **24 hours**.
    - **Subject**: Enter `repo:my-org/my-repo:ref:refs/heads/main` and replace the following placeholders.
        - **my-org**: your GitHub organization name
        - **my-repo**: the repository name you will create in Step 2
7. Select **Save**.

After saving, the **Client Authentication** section displays a **Client ID**. Note this value, you will need it as the `SCI_CLIENT_ID` repository variable in Step 4.

For background information, see the Cloud Identity Services documentation on JWT bearer flows.

Define a Dependency to the Joule Studio Application

1. Still on the **Trust** tab, under **Application APIs**, select **Dependencies**.
2. Select **Add** and fill in the dialog to add a dependency to the Joule Studio application:
    - **Dependency Name**: Enter **build**.
    - **Application**: Select **Build Core SAP Managed IAS (engagement layer)**
    - **API**: Select **solution-deployment**.
3. Select **Save**.

For background information, see the Cloud Identity Services documentation on integrating applications.

ℹ Note

Security note: This configuration allows GitHub workflows running on the main branch of your repository to call the Joule Studio solution management API. Limit repository access accordingly.

Step 2: Create a GitHub Repository

Create one empty, private GitHub repository for the solution you want to deploy to production.

1. Log in to GitHub and select **New repository**.
2. Set the **Owner**.
3. Enter a **Repository name** that identifies the solution.
4. Set **Visibility** to **Private**.
5. Leave all other options at their defaults (no template, no README, no .gitignore, no license).
6. Select **Create repository**.

Step 3: Set Up the GitHub Actions Workflow

In the new repository, create a GitHub Actions workflow file that handles production deployment.

1. In the repository, go to **Actions** and select the link **set up a workflow yourself**.
2. Name the action appropriately (e.g. `deploy.yml`).
3. Copy the workflow template provided by SAP at github.com/SAP/joule-actions into the **Edit** window for your action.
4. Select **Commit changes...** to add the file to the repository.

The workflow reads its configuration from repository variables (see Step 4) and uses the Joule Actions provided by SAP to package and deploy the solution.

Step 4: Configure Repository Variables

The following example workflow reads three variables from the repository. Set them under **Settings** > **Secrets and variables** > **Actions** > **Variables**:

Variable	Description
`SCI_TENANT_URL`	Issuer URI of your SAP Cloud Identity Services tenant, for example `https://<your-tenant>.accounts.ondemand.com`
`SCI_CLIENT_ID`	Client ID of the SCI application you created in Step 1
`SOLUTION_HANDLING_API_BASE_URL`	Base URL of the Joule Studio solution management API for your landscape

This information comes from IAS.

Setup is complete. The workflow is ready to run once you push a solution to the repository.

Connecting a Solution to GitHub

Once setup is complete, developers can connect individual solutions to the GitHub repository from within Joule Studio.

Prerequisites

You must have read and write permissions to the repository.

Procedure

1. Open your solution in Joule Studio.
2. Open the **More options** menu (⋯).
3. Choose **Connect to GitHub**.
4. Enter your GitHub repository URL.

   > **Note**
   > If you are using GitHub Enterprise, make sure your administrator has completed the GitHub Enterprise Setup first.

5. Choose **Connect to GitHub**.
6. Authenticate your user.

Once connected, the repository is associated with your solution. You cannot change the connected repository later.

The **Connect to GitHub** dialog then provides the following options:

- **Push to GitHub**: Sync your solution changes to the connected repository.
- **Pull**: Pull repository changes into Joule Studio.

Deploying to Production

Production deployment follows a two-phase process:

Phase 1: Deploy to development (in Joule Studio)

Deploy and validate your solution in the development environment using the Deploy button in Joule Studio. See Deployment for details.

Phase 2: Promote to production (via GitHub)

1. In Joule Studio, connect the solution to your GitHub repository and push the solution code to the repository directly from Joule Studio.
2. Inform your administrator that the solution is ready for production.
3. The administrator goes to the repository in GitHub and manually triggers the deploy workflow. This manual trigger is the approval gate for production.
4. Behind the scenes, the workflow packages the source code and calls the Joule Studio solution management API. No direct interaction with the managed runtime APIs is required.
5. Once the workflow completes, the agent is live in the productive environment. Deployment status is visible directly in GitHub Actions.

Administrators can also view the solution, its deployments, and observability data in the Joule Studio production environment.

Further Information

- Role Management: Confirm that administrators have the access needed to trigger production deployments.
- Deployment: Details on the development deployment flow in Joule Studio.

GitHub Enterprise Setup for Joule Studio

This page is only required for organizations using **GitHub Enterprise**. If your organization uses GitHub.com, no administrator setup in SAP for Me is needed — skip this page and go directly to GitHub Setup.

Configure your SAP for Me system to enable users to connect their Joule Studio solutions to your GitHub Enterprise instance.

> **Note**
> This guide assumes Joule Studio is already provisioned. If not, see Provisioning first.

Prerequisites

Before configuring GitHub Enterprise integration, ensure the following:

- **Joule Studio is already provisioned**: A formation is automatically created during provisioning.
- **Administrator access to SAP for Me**
- **Your GitHub Enterprise instance is accessible on the public network** (not behind a VPN)

Configuration Overview

The setup consists of four main steps:

1. Create a GitHub OAuth App
2. Request GitHub system type in SAP for Me
3. Create a GitHub system
4. Include the system in the Joule Studio formation

Step 1: Create a GitHub OAuth Application in GitHub Enterprise

The OAuth app allows Joule Studio to authenticate users into the GitHub Enterprise system (according to the user's existing permissions).

1. Log into your GitHub Enterprise instance.
2. Follow the steps described in the Creating an OAuth app topic in the GitHub documentation following these guidelines:
    - In the **Homepage URL** and the **Authorization callback URL**, you can use the organization URL (for example, `https://www.sap.com`), or any other valid URL.
    - Create the app under an official organization in the GitHub Enterprise, and not under your personal user account.
    - Make sure you select the **Enable Device Flow** checkbox before you register the application.
3. Once you have successfully registered the application, note the **Client ID** at the top of the page, you'll need this for the system configuration.

Step 2: Request a GitHub System Type

1. Log into **SAP for Me** as an administrator.
2. Navigate to **Portfolio and Products** and select the **My Product Packages** tab.
3. Select **Customer Landscape** → **Consent Requests**.
4. In the **Request Consent** dialog, enter:
    - **Request Consent For**: **System Type**
    - **Namespace**: `sap.github.github` (must be unique - only one per account)
    - **Scopes**: **System Type Sharing**
5. Choose **Request**.
6. Wait for SAP approval. You can see the approval status in the **For System Types** tab.

Step 3: Create a GitHub System

1. In **SAP for Me**, go to **Portfolio and Products** and select the **My Product Packages** tab.
2. Select **Customer Landscape** → **Systems** → **Other Systems**.
3. Choose **Add**.
4. In the **Add System** dialog, enter:
    - **System Type**: The system type you created.
    - **URL**: Your GitHub Enterprise URL (e.g., https://github.<domain_name>). Use only the domain (not the repository URL)
    - **Name**: An identifier for your GitHub enterprise system. This should be unique.
    - **System ID**: An identifier for your GitHub enterprise system, preferably, the same used in the **Name** field.

Step 4: Include the GitHub System in Formation

1. In **SAP for Me**, go to **Portfolio and Products** and select the **My Product Packages** tab.
2. Select **Customer Landscape** → **Formations**.
3. Select the relevant formation that was created during provisioning from the drop-down list. For example, **Integration with Joule Studio**.
4. Choose **Include Systems**.
5. Select the GitHub type you created.
6. Select the GitHub systems you want to include.
7. Enter the **Client ID** from your GitHub OAuth application.
8. Check the status until it changes to **Ready**.

Next Steps

Once the GitHub system is included in the formation, complete the remaining setup in GitHub Setup: configure SCI trust, create a GitHub repository, and set up the GitHub Actions workflow for production deployment.

Key Concepts

Joule Work and Joule Studio

Term	Definition
Joule	SAP's interface for all things AI that creates a seamless experience across SAP and non-SAP systems and works across business functions.
Joule Studio	A unified development environment for building and running AI agents, intelligent apps, automations, and workspaces, with the governance, security, and business understanding required to operate at scale.
Joule Work	The user experience component of Joule that provides multiple interaction methods with Joule Assistants and Joule Agents, enabling users to unify and execute tasks across the SAP software portfolio and third-party applications.
Joule Agents	The group of AI agents that reliably perform specific multi-step tasks to deliver decision support and speed up business workflows. Joule Agents are accessed with Joule Assistants.
SAP Business AI Platform	Brings together all of SAP's AI, data, processes, and AI governance capabilities into one foundation to create and integrate agents and applications grounded in business content.

Intent-Based Development Workflow in Joule Studio

Term	Definition
Intent	The first step of the development workflow where Joule Studio captures and validates your business challenge through clarifying questions and AI-driven discovery.
Quick Create	A mode in the Joule Studio web UI that skips the clarifying questions phase and starts building the solution directly. Use it when you already have a well-defined intent. When starting from the dialog form, leave **Quick Create** unchecked to go through the clarifying questions instead. When starting from the chat window, select **Quick Create** and choose **Ideate and plan** from the dropdown to refine your intent before building begins.
Requirements	The second step where Joule Studio generates a Product Requirements Document (PRD) based on validated intent analysis.
Specification	The third step where Joule Studio translates approved requirements into an implementation blueprint including architecture, data model, and task breakdown.
Solution	The fourth step where Joule Studio generates production-ready code based on the approved specification.
Testing	The fifth step where solutions are validated through the preview interface prior to production deployment.
Deployment	The sixth step where Joule Studio packages and deploys the solution to the managed runtime service.

AI-Driven Discovery and Solution Proposal

Term	Definition

Term	Definition
AI-Driven Discovery	The automated pipeline in Joule Studio that queries SAP intelligence systems to understand your landscape, map challenges to SAP architecture, and identify optimal solution approaches.
SAP LeanIX	SAP's enterprise architecture management platform that provides visibility into an organization's application landscape, business capabilities, interfaces, IT components, and ongoing initiatives.
SAP Knowledge Graph	SAP's enterprise knowledge system that provides API discovery, fit-gap analysis, and reference business architecture mapping.
SAP Domain Models	SAP's AI reasoning layer, trained on SAP application knowledge, business process expertise, and industry content. Enables accurate, SAP-specific guidance rather than generic AI responses.
SAP Reference Business Architecture	SAP's canonical framework organizing business processes into End-to-End processes, phases, sub-processes, and business activities.
Fit-Gap Analysis	The process of evaluating how well standard SAP assets cover business requirements, identifying what can be reused versus what requires custom development.
End-to-End Process	The highest level in SAP Reference Business Architecture (e.g., Source to Pay, Lead to Cash, Recruit to Retire).

Solution Types

Term	Definition
Joule Agent	An AI assistant that understands natural language, accesses SAP systems through a secure gateway, and delivers answers or takes actions through conversation.
Custom Agent	An agent built from scratch in Joule Studio to address a specific business challenge not covered by existing SAP-delivered agents.
LoB Agent	A Line of Business agent delivered by SAP for specific business domains (e.g., AP Invoice Agent).
Solution Category	The type of solution recommended by Joule Studio: SAP Product or Agent.

Documents & Artifacts

Term	Definition
Intent Document	The output of AI-driven discovery containing the business challenge, reference business architecture mapping, fit-gap analysis, and solution recommendation.
Idea Board	The screen in Joule Studio where the proposed solution appears after intent analysis. Shows the intent summary, problem statement, business context, recommended solution category (such as an agent), and an intent fit percentage.
PRD (Product Requirements Document)	A structured document containing user personas, functional requirements, API references, and success criteria for a solution.
Specification	A document containing the architecture design, data model, API integration plan, implementation tasks, and security model.

Protocols & Integration

Term	Definition
Agent Gateway	The secure access layer that serves as the single, governed connection point between deployed agents and SAP backend systems. Agents interact with SAP APIs through MCP (Model Context Protocol) tools, and the Agent Gateway ensures all requests are authenticated and compliant.
MCP (Model Context Protocol)	The protocol that agents use to interact with SAP APIs at runtime. Each API is exposed as an MCP tool that the agent can invoke, rather than calling HTTP endpoints directly. This ensures governed, structured access to backend systems.
SAP BTP Destination	A configured, secure connection in SAP BTP that provides managed access to SAP systems and external services. Set up once by an administrator; reused by all solutions that need it.

Runtime & Infrastructure

Term	Definition
Managed Runtime Service	The managed cloud platform on SAP BTP where Joule Studio solutions are deployed. SAP handles all infrastructure, scaling, updates, and operations - no server management required.

Pro-Code Development

Term	Definition
Pro-Code Development	The terminal-based way of building Joule Studio solutions, using the `jl` CLI and (optionally) an AI coding agent. Follows the same six-step workflow as the UI experience.
`jl` CLI	The command-line interface for Joule Studio. Used to scaffold, validate, build, and deploy solutions, and to manage authentication and AI coding agent integration.
AI Coding Agent	A terminal-based or IDE-integrated assistant (such as Claude Code or OpenCode) that drives the Joule Studio development workflow on your behalf, using skills and MCP tools fetched from the backend.
Skill (CLI)	A reusable prompt registered as a slash command in your AI coding agent. Each skill guides the agent through one phase of the Joule Studio workflow. Skills are backend-provided and may differ between landscapes.
Solution (Pro-Code)	A folder on disk containing a `solution.yaml` manifest and an `assets/` directory. Created with `jl solution create`.
Solution Asset	A typed component inside a solution (for example an agent or MCP server). Created with `jl asset create`.
MCP Server (asset type)	A pro-code asset that maps an existing API to MCP tools so that agents can invoke it. An MCP Server asset does not contain server implementation code. It provides the tool definitions and configuration that connect the agent to the API.
Fast Track	An accelerator in pro-code development for users who already have a well-defined intent. Adding "Fast track." to your prompt bypasses the interactive clarifying questions phase and moves directly to specification and code generation. All workflow phases still run. The difference is that intent refinement happens upfront in your prompt rather than through dialogue. For the equivalent behavior in the web UI, see ***Quick Create***.

Joule Studio for Web UI: Create an Agent in Joule Studio

Overview

This tutorial guides you through building your first agent in Joule Studio, following the complete development flow from intent to deployment.

Who is this for? Business users, citizen developers, and anyone building solutions with Joule Studio.

Time to complete: 15-20 minutes

What you will build: An agent that retrieves the top 5 cost centers from SAP S/4HANA and returns them in a conversational response.

Prerequisites

For this tutorial, make sure you have:

- Access to Joule Work (request access through your organization's CAM profile).
- An SAP S/4HANA system configured with an SAP BTP destination.
- Appropriate authorizations for the cost center data you want to access.

Procedure

Step 1: Open Joule Studio

1. Open Joule Work.
2. In the left navigation, select **Develop** to open Joule Studio.

You're now ready to create a new solution.

Step 2: Create a New Agent

1. Select **+ New** and then **Create** for the Agent solution type.
2. In the dialog, enter:
    - **Name**: a name for your solution (for example, `Cost Center Agent`).
    - **Intent Statement**: the prompt that describes what you want to build. For this tutorial, use:

```
Create an agent for Finance Controllers that queries cost center data in SAP S/4HANA Controll
delivering instant answers on total count of cost centers and top 5 cost centers, so that
controllers can answer department questions without running manual SAP transactions.
```

3. Make sure **Quick Create** is unchecked. Quick Create skips the clarifying questions and starts building the solution directly. For this tutorial, you want the clarifying questions so you can shape the intent.

   If you are starting from the chat window at the bottom of the dialog instead, select **Quick Create** and then choose **Ideate and plan** from the dropdown. This mode refines your intent before building begins, which is the equivalent of leaving Quick Create unchecked in the form above.

4. Select **Create**.

Joule Studio opens the new solution and asks clarifying questions to understand your requirements. Answer the questions based on your specific business context. There are no wrong answers during this exploration phase.

While you answer questions, Joule Studio performs the following AI-driven discovery in the background:

- Discovers your SAP landscape via SAP LeanIX
- Identifies relevant APIs from the SAP Knowledge Graph
- Performs fit-gap analysis to determine what requires custom development
- Maps your intent to SAP processes using the SAP Reference Business Architecture

When discovery completes, your planned solution appears on the **Idea Board**, with an ***intent fit*** score and a recommended solution category.

Step 3: Review and Refine Your Intent

Use the **Idea Board** to review the proposed solution:

- **View** mode: See the solution overview and refine using the Joule chat.
- **Design** mode: Configure specific settings and parameters.
- **Code** mode: Edit the code of artifacts directly.

When you are satisfied with the intent, select **Create Requirements** or type "Create PRD" in the Joule chat.

Step 4: Review the Requirements

Joule Studio generates a Product Requirements Document (PRD) that captures:

- Value proposition and business context
- User personas and goals
- Functional and non-functional requirements
- Solution architecture

Review the PRD to ensure it accurately reflects your needs. Use the Joule chat to request changes if needed.

When the requirements are complete, select **Build > Build Specification** or type "Create Specs" in the Joule chat.

Step 5: Review the Specification

Joule Studio translates the PRD into a technical specification that defines:

- Architecture and component structure
- Data model and API integrations
- Implementation tasks
- Security model

The specification references relevant SAP APIs from the SAP Business Accelerator Hub. Review the technical approach and request changes if needed.

When the specification is complete, select **Build** or type "Create Solution" in the Joule chat.

Step 6: Review the Generated Solution

Joule Studio generates a ready-to-deploy solution based on the specification. For this agent, the generated solution includes:

- The agent's behavior and the business logic it will apply
- Secure connections to the relevant SAP S/4HANA APIs
- Error handling to manage unexpected results gracefully

Review the solution in Joule Studio. Use the Joule chat to request changes, for example to adjust which data the agent retrieves or how it formats its response.

Step 7: Test the Solution

Switch to **Testing** in the progress bar to view solution tests. Joule Studio generates tests that verify:

- Code correctness for API integrations
- Response formatting
- Error handling

In addition to standard code tests, Joule Studio generates AI-powered evaluations that assess the agent's conversational behavior and response quality. Review these evaluation results alongside the technical test results.

Review the test results. If tests fail, use the Joule chat to iterate on the implementation.

Step 8: Deploy the Solution

When testing is complete, select **Deploy**.

Joule Studio packages and deploys the solution to the managed runtime service. The deployment process:

- Packages the solution with all dependencies
- Provisions the runtime environment
- Registers the agent with the required SAP services

- Activates monitoring and logging

Monitor the deployment progress in the log panel. When deployment completes, the agent is available in Joule.

Step 9: Test the Deployed Agent

! Restriction

Before you can use the deployed agent, you need to be granted access to it in the productive landscape. This is done by your administrator, following the process described in Assign Agent Access. Please be aware that this process is not fully available yet, so the following steps cannot be performed on the productive landscape.

After deployment completes, confirm that the agent works correctly with your live SAP data.

1. In Joule Studio, when you're on **Deployment**, open the latest deployment (if there are more than one), and select **Try now** to open the agent.
2. Send the agent a test query that exercises your use case, for example: "Show me the top 5 cost centers."
3. Verify that the agent returns results drawn from your SAP S/4HANA system.

Step 10: Check Monitoring and Traces

Confirm that your test activity was recorded.

1. To inspect the trace from your test query in detail, go back to the **Solution** page and switch to **Try** mode. Select the **Traces** panel below the chat to see the tool calls and reasoning steps from your test run.
2. To check aggregate runtime metrics, open the **Solution Overview** tab and select the **Monitoring** tab. This view shows session and workflow counts over time, not individual trace detail.

If no data appears in the Monitoring tab yet, wait a few minutes -- monitoring data can take a short time to propagate.

Managing Your Solution

Your solutions are listed in the left panel of Joule Studio. Selecting a solution opens an overview in the center area, showing its description, creation details, and artifacts. To open the solution and work with its content, select **Open** in the top-right corner.

To manage a solution at any time, select the **More options** button (the three-dot icon) in the top-right corner of any page within the solution. This opens a menu with the following options:

- **Rename**: Change the name of your solution.
- **Connect to GitHub**: Link your solution to a GitHub repository. Once connected, you can push your solution files to GitHub directly from Joule Studio, or pull updates from the repository. Use this to keep your solution under version control and to enable production deployment.
- **Download ZIP**: Download the complete solution files as a ZIP archive. You can use this to back up your solution or share it manually with a colleague.
- **Undeploy**: Reverse a deployment and remove the active version of your solution from the development environment. The solution itself is not deleted and can be redeployed at any time.

- **Delete**: Permanently delete the solution and all its content. This action cannot be undone.

Next Steps

Now that you have built your first agent, explore the following topics:

- AI-Driven Discovery: Learn how Joule Studio analyzes your intent and proposes solutions
- Solution Types: Understand the different types of solutions you can build
- Testing: Learn about generated tests and AI-powered evaluation
- Deployment: Understand the deployment process and runtime capabilities

Example Intent Statements

Use these examples as inspiration for your next solution:

Domain	Intent Statement
Sales	I need an agent to suggest discount strategies based on the customer's order and margin targets.
HR	I want an onboarding agent that answers employee questions about onboarding steps and company policies.
Finance	Build an agent that drafts personalized collection emails based on payment history and outstanding items.
Procurement	I need an agent that summarizes supplier performance trends and recommends alternatives from the approved vendor list.

Tips for Writing Effective Intents

- **Be specific about the business problem**: Describe what you are trying to accomplish, not just what you want to build.
- **Mention the data sources**: Specify which SAP systems contain the data you need.
- **Include success criteria**: Describe what a successful outcome looks like.
- **Start simple**: Begin with a focused use case and expand after validating the initial solution.

Joule Studio for CLI: Pro-Code Development in Joule Studio

What Is Pro-Code Development?

Pro-code development is the way professional developers build solutions in Joule Studio from their preferred IDE. Instead of working in the browser-based Joule Studio UI, you use the `jl` command-line interface (CLI) to scaffold, validate, build, and deploy solutions. You can drive the workflow yourself, or pair the CLI with an AI coding agent such as Claude Code or OpenCode to handle the conversational flow.

Pro-code development follows the same intent-based development workflow as the UI experience. Both paths produce the same artifacts (PRD, specification, solution code) and deploy to the same managed runtime service. What differs is the surface: the UI is optimized for business users and citizen developers; the CLI is optimized for professional developers who want full control over the code, file structure, and toolchain.

Who Is the Pro-Code Workflow For?

Pro-code development is intended for:

- Professional developers who are comfortable working in a terminal and an IDE.
- Developers who want to integrate solution builds into existing CI/CD pipelines.
- Anyone who needs access to asset types that are not exposed in the UI, such as MCP servers, frontend applications, or domain model extensions.

If you are a business user or citizen developer, the UI-based experience in Joule Studio is the better fit. See Getting Started: Create an Agent in Joule Studio.

Access

You access pro-code development by installing the `jl` CLI from the internal SAP package registry. You also need an account on your target Joule Studio landscape and, optionally, an AI coding agent such as Claude Code or OpenCode.

For the complete setup instructions, see Pro-Code Setup.

Getting Started

To learn how to build your first solution from the terminal, see Getting Started: Create an Agent with the CLI.

Key Advantages of Pro-Code Development

- Full control over solution code, file structure, and toolchain in your local IDE.
- Scriptable end-to-end flow for CI/CD pipelines and automated testing.
- AI-assisted development via Claude Code or OpenCode, using skills and MCP tools fetched from the Joule Studio backend.

- Access to asset types that are not available in the UI, such as `mcp-server`, `base-ui`, `vercel-app`, `data-product`, and `domain-model-extension`.

Pro-Code vs. UI Development

Both paths build the same kinds of solutions and deploy to the same runtime. Use this table to choose the right path for your work.

Aspect	UI (Joule Studio in Joule Work)	Pro-Code (`jl` CLI)
Primary audience	Business users, citizen developers	Professional developers
Where you work	Browser	Terminal and local IDE
How you describe intent	Joule chat in the UI	Prompts to an AI coding agent, or direct CLI commands
CI/CD integration	Limited	Full (CLI is scriptable)
Available asset types	Agents	Agents, plus additional pro-code-only asset types
Workflow	Intent → Requirements → Specification → Solution → Testing → Deployment	Same six steps, driven by CLI commands and the AI coding agent

Asset Types in Pro-Code

Pro-code development supports the same asset types as the UI, plus additional ones that give developers finer-grained control over solution composition.

Asset Type	Description	Example
Agents	AI agents that answer questions or take action by connecting to SAP systems.	Missed Discount Analysis Agent

For asset types specific to pro-code (such as `mcp-server` and `base-ui`), see Solution Structure. For an overview of solution types, see Solution Types.

Workflow

Pro-code development follows the same six-step intent-based development workflow as the UI. The first three steps (intent, requirements, and specification) are driven by an AI coding agent such as Claude Code or OpenCode. The later steps (solution, testing, and deployment) are driven from the CLI directly, with optional support from the coding agent.

1. Intent

Describe your business challenge in a prompt to your AI coding agent. The agent asks clarifying questions and runs the AI-driven discovery against your SAP landscape.

2. Requirements

The AI coding agent produces a Product Requirements Document (PRD) capturing user personas, requirements, scope, and architecture.

3. Specification

The PRD is broken down into a technical specification with architecture decisions, API references, data models, and a prioritized task list.

4. Solution

The AI coding agent generates the solution code, including the `solution.yaml` manifest and the agent files.

5. Testing

You validate the solution locally with `jl solution validate` and then test the deployed solution in your sandbox landscape.

6. Deployment

You build the solution into a deployable archive with `jl solution build` and deploy it with `jl solution deploy`.

For details on each step and the corresponding CLI commands, see Pro-Code Workflow.

AI-Driven Solution Discovery

Pro-code development uses the same AI-driven solution discovery as the UI. When you describe your intent, the system queries SAP LeanIX, the SAP Knowledge Graph, SAP Domain Models, and runs a fit-gap analysis to ground the proposed solution in your actual SAP landscape.

For details on the discovery process, see Intent Refinement with AI-Driven Discovery.

Related Information

- Getting Started: Create an Agent with the CLI
- Pro-Code Setup
- AI Coding Agent
- Pro-Code Workflow
- Solution Structure
- CLI Reference

Placeholder Notes

The following placeholder is used in this file. Replace with the final wording before publishing:

- `internal SAP package registry`: used in the Access section. Replace with the public-facing wording you want to use for the package source.

Joule Studio for CLI: Create an Agent with the CLI

Overview

This tutorial guides you through building your first agent in Joule Studio from the terminal, using the `jl` CLI and an AI coding agent. It is the pro-code counterpart to Getting Started: Create an Agent in Joule Studio and follows the same six-step intent-based development workflow.

Who is this for? Professional developers who prefer working in a terminal and an IDE.

Time to complete: 20-30 minutes

What you will build: An agent that helps a finance team identify and follow up on supplier invoices that have not been matched against purchase orders.

Prerequisites

Before you begin, make sure you have:

- Node.js version 24 or later installed.
- Access to a Joule Studio landscape (request access through your organization's access management process).
- An SAP S/4HANA system configured with an SAP BTP destination.
- Appropriate authorizations for the purchase order and supplier invoice data you want to access.
- An AI coding agent installed locally, such as Claude Code or OpenCode.

Procedure

Step 1: Install the CLI

Install the `jl` CLI globally with npm:

```
npm install -g @sap/joule-work-dev-cli-early-adopters@prerelease
```

ℹ Note

This is the command during the early adopter care phase of Joule Studio and will change once Joule Studio is generally available.

ℹ Keep the CLI up to date

There is no automatic update notification. Re-run the install command periodically to pick up the latest version.

Verify the installation:

```
jl v
```

For full setup details, see Pro-Code Setup.

Step 2: Log In to Your Landscape

Before you can log in from the CLI, get your personal Sign-in URL from Joule Studio:

1. Open Joule Studio in your browser.
2. Select your user icon in the bottom-left corner and choose **Settings**.
3. Go to **Develop**.
4. Under **CLI Authorization**, copy the **Sign-in URL**.

Pass the URL to `jl login`. Wrap it in single quotes so the shell does not split it at the & characters:

```
jl login '<your Sign-in URL>'
```

A browser window opens for you to authenticate. Once you confirm, the CLI stores the session and you can close the browser tab. After the first successful login, running `jl login` without an argument reuses the stored URL.

Step 3: Initialize Your Project

Create a folder for your project and initialize it for AI coding agent integration. The folder name can be anything DNS-compatible. This tutorial uses `po-invoice-matching-agent` as an example.

```
mkdir po-invoice-matching-agent && cd po-invoice-matching-agent
jl init claude
```

Replace `claude` with `opencode` if you are using OpenCode instead.

This command fetches the latest Joule Studio skills from the backend and writes the MCP server configuration for your AI coding agent. The skills appear as slash commands inside your AI coding agent.

ℹ Note

Note: If you use Visual Studio Code, open the project folder as the workspace root so that the `.mcp.json` file is at the top level. After running `jl init`, reload the window so that the AI coding agent picks up the new configuration.

Step 4: Describe Your Intent

Open your AI coding agent in the project folder and describe the solution you want to build. For this tutorial, use the following prompt:

```
Our finance team spends hours every week manually matching purchase orders with incoming supplier i
```

The AI coding agent asks clarifying questions to understand your requirements. Answer the questions based on your specific business context.

While you answer, the agent runs the same AI-driven discovery as the UI experience:

- Discovers your SAP landscape via SAP LeanIX
- Identifies relevant APIs from the SAP Knowledge Graph
- Performs fit-gap analysis to determine what requires custom development
- Maps your intent to SAP processes using the SAP Reference Business Architecture

ⓘ Note

Tip: Add **"Fast track."** at the end of your prompt to skip the clarifying questions and jump directly to specification and code generation.

Step 5: Review the Requirements

Working with your AI coding agent, produce a Product Requirements Document (PRD) that captures:

- Value proposition and business context
- User personas and goals
- Functional and non-functional requirements
- Solution architecture

Review the PRD with your AI coding agent and ask for changes if needed.

Step 6: Review the Specification

Working with your AI coding agent, translate the PRD into a technical specification that defines:

- Architecture and component structure
- Data model and API integrations
- Implementation tasks
- Security model

The specification references relevant SAP APIs from the SAP Business Accelerator Hub. Review the technical approach and ask for changes if needed.

Step 7: Review the Generated Solution

The AI coding agent generates the solution code in your project folder, including:

- A `solution.yaml` manifest at the project root
- An `assets/` directory containing the generated assets
- The agent's behavior and business logic for matching purchase orders with supplier invoices
- Secure connections to the relevant SAP S/4HANA APIs

- Error handling for unexpected results

For an explanation of the project layout, see Solution Structure.

Review the generated files in your IDE. Use the AI coding agent to request changes, or edit the files directly.

Step 8: Validate the Solution

Run a local validation pass to catch structural problems before you build:

```
jl solution validate
```

The validation checks that `solution.yaml` exists, that all referenced assets resolve to real files, and that each asset passes its plugin-specific checks. Fix any reported errors before continuing. For the full list of validation rules, see CLI Reference.

Step 9: Build and Deploy the Solution

Deploy the solution to your landscape. The command runs a full `create → validate → build → deploy` cycle automatically:

```
jl solution deploy
```

If you prefer to build and deploy as separate steps, you can run them individually:

```
jl solution build
jl solution deploy ./build/<your-solution-name>-<version>.zip
```

Monitor the deployment progress:

```
jl solution deploy status
```

If deployment fails or you want to inspect runtime logs, use:

```
jl solution deploy logs
```

Verify Your Deployed Agent

After deployment, verify that the agent is reachable:

1. Check the deployment status with `jl solution deploy status` and confirm that the deployment completed successfully.
2. List your solutions with `jl solution list` to see the registered agent.
3. Interact with the deployed agent in one of the following ways:
    - Through the Agent-to-Agent (A2A) protocol from the CLI:

      ```
      jl a2a message "Show me supplier invoices that have not been matched to a purchase orde
      ```

- Through Joule Work, once the agent is registered as a Joule Capability. When users ask matching questions in Joule Work, Joule routes the request to your agent automatically.

Next Steps

Now that you have built your first agent from the CLI, explore the following topics:

- AI Coding Agent: Learn how skills and MCP integration support the development flow.
- Pro-Code Workflow: See how the six-step workflow maps to CLI commands and AI-agent prompts.
- Solution Structure: Understand the project layout and asset types.
- CLI Reference: Look up every available `jl` command.

Tips for Effective Pro-Code Development

- **Start with a clear intent**: The clarifying questions are easier to answer when the business problem is well defined.
- **Keep solutions under version control**: Commit `solution.yaml` and the `assets/` directory; ignore the generated `build/` output.
- **Use Fast track when you have a well-defined intent**: It moves directly to specification and code generation, so the clarifying questions that surface hidden assumptions do not run.
- **Validate often**: Run `jl solution validate` after every significant change so problems surface early.
- **Refresh skills periodically**: Run `jl init claude --force` (or `jl init opencode --force`) to pick up new or updated skills from the backend.

Placeholder Notes

The following placeholders are used in this file. Replace with the final wording before publishing:

- "your organization's access management process": replace with the public-facing access path.

Pro-Code Setup

This page walks you through setting up your local environment for pro-code development with Joule Studio. You install the `jl` CLI, log in to your landscape, and prepare your project folder for use with an AI coding agent.

For a guided first build after setup, see Getting Started: Create an Agent with the CLI.

Prerequisites

Before you start, make sure you have:

- Node.js version 24 or later installed.
- An account on your target Joule Studio landscape. Request access through your organization's access management process.
- An AI coding agent installed locally. Joule Studio currently supports Claude Code and OpenCode.

Install the CLI

Install the `jl` CLI globally with npm:

```
npm install -g @sap/joule-work-dev-cli-early-adopters@prerelease
```

ℹ Note

This is the command during the early adopter care phase of Joule Studio and will change once Joule Studio is generally available.

ℹ Keep the CLI up to date

There is no automatic update notification. Re-run the install command periodically to pick up the latest version.

Verify the installation:

```
jl v
```

The command prints the installed version. If the command is not found, make sure your global npm bin directory is on your PATH.

Log In to a Landscape

Before you can log in from the CLI, get your personal Sign-in URL from Joule Studio:

1. Open Joule Studio in your browser.
2. Select your user icon in the bottom-left corner and choose **Settings**.
3. Go to **Develop**.

4. Under **CLI Authorization**, copy the **Sign-in URL**.

Pass the URL to `jl login`. Wrap it in single quotes so the shell does not split it at the & characters:

```
jl login '<your Sign-in URL>'
```

A browser window opens for you to authenticate. Once you confirm, the CLI stores the session and you can close the browser tab. After the first successful login, running `jl login` without an argument reuses the stored URL.

To log out and clear the stored session:

```
jl logout
```

Initialize Your Project

Create a folder for your solution and initialize it for AI coding agent integration:

```
mkdir my-solution && cd my-solution
jl init claude
```

Replace `claude` with `opencode` if you are using OpenCode instead.

This command:

- Fetches the latest Joule Studio skills from the backend and stores them locally:
  - `.claude/skills/` for Claude Code
  - `.agents/skills/` for OpenCode
- Writes the configuration files that connect your AI coding agent to Joule Studio:
  - `.mcp.json`: shared MCP server configuration.
  - `.claude/settings.local.json`: Claude Code settings, including MCP permissions.
  - `opencode.json`: OpenCode configuration.

ℹ Note

Visual Studio Code: Open the project folder as the workspace root so that the `.mcp.json` file sits at the top level. After running `jl init`, reload the window so that the AI coding agent picks up the new configuration.

If you need to refresh skills or overwrite the existing configuration, run:

```
jl init claude --force
```

Verify Your Setup

After `jl init`, verify that the AI coding agent recognizes the Joule Studio integration. For Claude Code, list the registered MCP servers:

```
claude mcp list
```

You should see a `joule-studio` entry. Then, inside Claude Code, type `/` to see the list of skills fetched from the backend.

What Are Skills?

Skills are slash commands registered in your AI coding agent. Each skill guides you through one phase of the Joule Studio development workflow, for example capturing intent, producing a PRD, or generating a task spec. Skills are provided by the Joule Studio backend and may differ between landscapes.

For more information, see AI Coding Agent.

Troubleshooting

Problem	Likely Cause	What to Try
`jl` command not found	Global npm bin directory not on PATH.	Add the directory printed by `npm bin -g` to your PATH.
`jl login` fails immediately	Wrong Sign-in URL or session already invalid.	Copy a fresh Sign-in URL from Settings > Develop in Joule Studio. If you logged in recently, run `jl logout` first and retry.
AI coding agent does not see Joule Studio	The agent is opened against a different folder than the one containing `.mcp.json`.	Open the folder containing `.mcp.json` as the workspace root and reload the window.
Skills do not appear after `jl init`	Skills not fetched, or your AI coding agent did not reload.	Run `jl init claude --force` (or `jl init opencode --force`) and reload your AI coding agent.

Related Information

- Pro-Code Development in Joule Studio
- Getting Started: Create an Agent with the CLI
- AI Coding Agent
- CLI Reference

Placeholder Notes

The following placeholders are used in this file. Replace with the final wording before publishing:

- "your organization's access management process": replace with the public-facing access path.

AI Coding Agent

What Is an AI Coding Agent?

An **AI coding agent** is a terminal-based or IDE-integrated assistant that drives the Joule Studio intent-based development workflow on your behalf. Instead of clicking through the UI to capture intent, generate a PRD, and produce code, you describe what you want in plain language, and the AI coding agent runs the workflow using skills and MCP tools provided by the Joule Studio backend.

Joule Studio currently supports two AI coding agents:

- **Claude Code**: Anthropic's official CLI for Claude.
- **OpenCode**: A general-purpose AI coding agent.

The role of the AI coding agent in pro-code development is similar to the role of the Joule chat in the UI experience: it is the conversational interface that captures intent, asks clarifying questions, and orchestrates code generation. The difference is that the AI coding agent runs locally, can read and write files in your project, and can integrate with your IDE.

How It Connects to Joule Studio

When you run `jl pro-code init` in a project folder, the CLI sets up two things that make the integration work:

1. **AI coding agent configuration**: Writes a set of configuration files that tell your AI coding agent how to talk to Joule Studio backend services through the Model Context Protocol (MCP):
    - `.mcp.json`: shared MCP server configuration.
    - `.claude/settings.local.json`: Claude Code settings, including MCP permissions.
    - `opencode.json`: OpenCode configuration.
2. **Skills**: Fetches the current set of Joule Studio skills from the backend and stores them under `.claude/skills/`. Each skill becomes a slash command inside your AI coding agent.

The AI coding agent uses the MCP tools at runtime to query SAP LeanIX, the SAP Knowledge Graph, and SAP Domain Models, and to access generation services that produce solution artifacts.

Skills

Skills are reusable prompts that guide the AI coding agent through one phase of the development workflow. They appear as slash commands in your AI coding agent (for example, `/intent` or `/spec`), and each one tells the agent how to perform a specific task in line with Joule Studio conventions.

Skills are backend-provided, which means:

- Their content is maintained centrally and reflects the current Joule Studio guidance.
- They may differ between landscapes (Joule-STAGING and Joule-DEV may publish different skill sets).
- They evolve over time. Refresh them when you start a new project or notice they are out of date.

To refresh skills, run:

```
jl pro-code init --force
```

This command re-fetches and overwrites the skills stored under `.claude/skills/`.

Inspecting MCP Servers Manually

The AI coding agent uses MCP tools automatically through `jl mcp start`. If you want to see what is available, the CLI also exposes the MCP layer directly:

- `jl mcp list`: list the MCP servers your landscape provides.
- `jl mcp tools <server>`: show the tools each server exposes.
- `jl mcp call <server> <tool> '<json>'`: invoke a tool from the terminal.

These commands are useful for exploration and debugging. Day-to-day, the AI coding agent calls them on your behalf.

What the AI Coding Agent Does for You

When you describe your intent, the AI coding agent typically:

1. Asks clarifying questions to understand pain points and success criteria.
2. Runs the AI-driven discovery against your SAP landscape (LeanIX, Knowledge Graph, Domain Models, fit-gap).
3. Helps produce a Product Requirements Document (PRD) capturing personas, requirements, and architecture.
4. Helps translate the PRD into a technical specification with architecture decisions and a task list.
5. Generates the `solution.yaml` manifest and asset files in your project folder.
6. Walks you through validation, build, and deployment, suggesting or running the corresponding `jl` commands.

The exact split between actions the AI coding agent performs automatically and actions you run yourself depends on the skills available in your landscape. You stay in control: you can review and modify any artifact at any step, ask the agent for changes in plain language, or edit the files directly in your IDE.

For details on the discovery process, see Intent Refinement with AI-Driven Discovery.

Fast Track

For straightforward use cases, you can use Fast track to move directly from your prompt to specification and code generation. Add **"Fast track."** to the end of your prompt:

```
Our finance team spends hours every week manually matching purchase orders with
incoming supplier invoices. It's error-prone and blocks payments. I need a
solution to automate that. Fast track.
```

All workflow phases still run. The difference is that intent refinement happens upfront in your prompt rather than through clarifying questions. Use Fast track when you already have a clear, well-scoped intent. For new or ambiguous use cases, the clarifying questions add value by surfacing assumptions early.

What You Do Yourself

The AI coding agent automates much of the conversational workflow, but some responsibilities remain yours:

- **Reviewing artifacts**: The PRD, specification, and generated code should always be reviewed before deployment.
- **Editing files directly**: For targeted code changes, editing in your IDE is often faster than describing the change to the agent.
- **Confirming CLI actions**: Validation, build, and deployment are CLI commands. The AI coding agent may suggest them or run them on your behalf, depending on the skill in use, but the decision to proceed is yours.
- **Version control**: Commit, branch, and review through your normal Git workflow.
- **CI/CD integration**: Wire `jl solution validate`, `jl solution build`, and `jl solution deploy` into your pipelines.

Driving the Workflow Without an AI Coding Agent

You do not need an AI coding agent to use pro-code development. You can drive the entire workflow with `jl` commands directly:

```
jl solution create --name my-solution
jl asset create agent --name my-agent
jl solution validate
jl solution build
jl solution deploy ./build/my-solution-1.0.0.zip
```

This direct approach is useful when you already know exactly what you want to build, when you are scripting solution generation in CI/CD, or when you prefer not to use an AI coding agent. The AI-driven discovery is not part of this manual flow.

For the full mapping between the workflow and CLI commands, see Pro-Code Workflow.

Related Information

- Pro-Code Setup
- Pro-Code Workflow
- Solution Structure
- CLI Reference
- Intent Refinement with AI-Driven Discovery

Pro-Code Workflow

Pro-code development follows the same six-step intent-based development workflow as the UI: Intent, Requirements, Specification, Solution, Testing, and Deployment. What changes is the surface. Each step is driven by your AI coding agent, by direct `jl` CLI commands, or by a combination of both.

This page maps each workflow step to the corresponding pro-code actions. For the conceptual description of each step, follow the linked pages.

Workflow at a Glance

Step	What You Do in Pro-Code	Key CLI Commands
Intent	Describe the business challenge in your AI coding agent. The agent runs clarifying questions and AI-driven discovery.	(driven by AI coding agent)
Requirements	The AI coding agent produces a PRD; you review and refine it.	(driven by AI coding agent)
Specification	The AI coding agent translates the PRD into a technical specification with a task list.	(driven by AI coding agent)
Solution	The AI coding agent generates `solution.yaml` and the assets; you can also create them directly.	`jl solution create`, `jl asset create`
Testing	Test the deployed solution in a sandbox landscape (`jl solution validate` is run before building, see Deployment).	`jl a2a message`
Deployment	Build the deployable archive and deploy it to your landscape.	`jl solution build`, `jl solution deploy`

Step 1: Intent

In the UI, you describe your intent in the Joule chat. In pro-code development, you describe it to your AI coding agent.

Example prompt:

```
Our finance team spends hours every week manually matching purchase orders with
incoming supplier invoices. It's error-prone and blocks payments. I need a
solution to automate that.
```

The AI coding agent asks clarifying questions and runs the AI-driven discovery against your SAP landscape (LeanIX, Knowledge Graph, Domain Models, fit-gap). The output is the same intent document produced by the UI experience.

ℹ Note

Tip: Add **"Fast track."** at the end of your prompt to move directly to specification and code generation. All workflow phases still run. The difference is that intent refinement happens upfront in your prompt rather than through dialogue.

For details on the discovery process, see Intent Refinement with AI-Driven Discovery.

Step 2: Requirements

Working with your AI coding agent, produce a Product Requirements Document (PRD) from the intent analysis. The PRD captures personas, functional and non-functional requirements, scope, and architecture.

Review the PRD with your AI coding agent and ask for changes if needed.

For the structure of the PRD and what to look for during review, see Requirements.

Step 3: Specification

Working with your AI coding agent, translate the approved PRD into a technical specification with architecture decisions, data models, API references, and a prioritized task list.

Review and refine the specification before generation begins.

For details, see Specification.

Step 4: Solution

In this step, the actual code is generated.

With an AI Coding Agent

The agent generates the `solution.yaml` manifest and the asset files based on the specification. After generation, your project structure looks roughly like this:

```
my-solution/
├── solution.yaml
└── assets/
    └── my-agent/
        ├── asset.yaml
        └── ...
```

For an explanation of the layout, see Solution Structure.

Without an AI Coding Agent

You can also create the solution manifest and assets directly from the CLI:

```
# 1. Scaffold the solution
jl solution create --name my-solution

# 2. Add the agent asset
jl asset create agent --name my-agent
```

This is useful for scripted use, CI/CD pipelines, or when you already know exactly which assets you need.

For details on solution generation and the asset types, see Solution and Solution Structure.

Step 5: Testing

Testing in pro-code development is a runtime test against the deployed solution. A separate validation step (`jl solution validate`) runs before the build to verify the structure of the solution and that the configured rules are valid. See "Local Validation" below.

Local Validation

Run the validation pipeline before building:

```
jl solution validate
```

Validation verifies the structure of the solution and that the configured rules are valid. Specifically, it checks that `solution.yaml` exists, that referenced assets resolve to real files, and that each asset passes its plugin-specific checks. Validation is the step before building. The command exits with code `0` on success and `1` on failure, so it integrates cleanly with CI/CD.

Runtime Testing

After deployment, you can interact with the deployed agent in one of the following ways:

- Through the Agent-to-Agent (A2A) protocol from the CLI:

  ```
  jl a2a message "What is the status?" --solution <your-solution-id> --asset <your-agent-name>
  ```

- Through Joule Work, once the agent is registered as a Joule Capability.

For more detail on testing strategy, see Testing.

Step 6: Deployment

Build the solution into a deployable archive:

```
jl solution build
```

The build produces a zip file at `build/<solution-name>-<version>.zip`. Deploy the archive to your landscape:

```
jl solution deploy ./build/my-solution-1.0.0.zip
```

Monitor the deployment:

```
jl solution deploy status
```

If something fails, inspect the logs:

```
jl solution deploy logs
```

To see all solutions you have deployed to the landscape, run:

```
jl solution list
```

For details on deployment outcomes and runtime capabilities, see Deployment.

Mapping CLI Commands to Workflow Steps

The following table is a quick reference that maps every key CLI command to the workflow step where you typically use it.

CLI Command	Workflow Step
`jl login`, `jl pro-code init`	(one-time setup)
(AI coding agent prompt)	Intent, Requirements, Specification
`jl solution create`	Solution
`jl asset create`	Solution
`jl solution validate`	(Solution validation)
`jl solution build`	Deployment
`jl solution deploy`	Deployment
`jl solution deploy status`, `jl solution deploy logs`	Deployment
`jl a2a message`	Testing

For the complete CLI command reference, see CLI Reference.

Related Information

- Pro-Code Development in Joule Studio
- Getting Started: Create an Agent with the CLI
- AI Coding Agent
- Solution Structure
- CLI Reference

Solution Structure

A pro-code solution is a folder on disk with a manifest file and one or more asset folders. This page explains the structure, the role of each file, and the asset types you can include.

Project Layout

A typical pro-code solution looks like this:

```
my-solution/
├── solution.yaml          ← solution manifest
├── assets/                ← all assets live here
│   ├── my-agent/
│   │   ├── asset.yaml
│   │   └── ...            ← asset-specific files
│   ├── my-service/
│   │   ├── asset.yaml
│   │   └── ...
│   └── my-workflow/
│       ├── asset.yaml
│       └── ...
├── build/                 ← created by `jl solution build` (do not commit)
└── .mcp.json              ← created by `jl pro-code init` (commit if shared with team)
```

You commit `solution.yaml`, the `assets/` directory, and (optionally) `.mcp.json` to Git. The `build/` folder contains generated archives and should be excluded from version control.

`solution.yaml`

`solution.yaml` is the manifest that defines the solution. It is created by `jl solution create` and lists every asset that the solution contains.

Required fields:

Field	Purpose
`apiVersion`	Schema version. Must be `solution.sap/v1`.
`kind`	Resource kind. Must be `Solution`.
`metadata.name`	Solution name. Must be DNS-compatible: lowercase letters, digits, and hyphens; 1 to 63 characters; starting and ending with a letter or digit.
`metadata.version`	Solution version (for example `1.0.0`).
`assets[]`	List of references to asset definitions in the `assets/` directory.

Run `jl solution validate` to check that all fields are present and that every asset reference resolves to a real file.

Optional fields you can add to `solution.yaml`:

Field	Purpose

Field	Purpose
`metadata.description`	Short human-readable description of the solution.
`metadata.labels`	Key-value labels for organizational use (string values).
`runtime.type`	Runtime type. Defaults to `generic`.
`requires[]`	Solution-wide dependencies inherited by all assets. Each entry can be a string or an object with `name` (required), `version`, `kind` (`service`, `platform-service`, or `asset`), and `plan`.
`overrides`	Solution-level configuration overrides applied to assets at runtime.
`custom`	Opaque platform-specific configuration (for example, properties consumed by Vercel, CAP, Cloud Foundry, or Kubernetes tooling).

For the full schema, see the JSON schema bundled with the CLI.

The `assets/` Directory

Each asset lives in its own subdirectory under `assets/`. The subdirectory name is the asset name and must match the `name` you pass to `jl asset create`.

Each asset folder contains:

- An `asset.yaml` envelope with required metadata.
- Asset-specific files (source code, configuration, workflow definitions, and so on).

You add assets with `jl asset create <type> --name <name>`. The CLI creates the folder, writes a starter `asset.yaml`, and updates `solution.yaml` to reference it. To remove an asset, use `jl asset delete <name>`, which deletes the folder and removes the reference.

Asset Types

The CLI supports the following built-in asset types.

Type	Description	Use When
`agent`	Agent with container and health probe configuration. Exposes `/.well-known/agent.json`.	You are building a new agent from scratch.

Asset Composition Examples

Goal	Typical Composition
Build a conversational agent	`agent`

Validation Rules

`jl solution validate` runs the following checks. Errors block the build; warnings do not.

#	Check	Severity
1	`solution.yaml` exists.	error
2	Required fields present (`apiVersion`, `kind`, `metadata.name`, `metadata.version`).	error
3	Manifest conforms to the JSON schema.	error
4	`assets/` directory exists.	error
5	Every `ref` in `assets[]` resolves to an existing file.	error
6	Every subdirectory in `assets/` contains an `asset.yaml`.	warning
7	Every `asset.yaml` has the required envelope fields.	error
8	Every asset passes its plugin-specific validation.	error

For typical fixes, see the validation table in CLI Reference.

Related Information

- Pro-Code Workflow
- CLI Reference
- Solution Types
- Solution

CLI Reference

This page documents every command of the `jl` CLI. Commands follow the pattern:

```
jl <command> [<subcommand>] [<args>] [<options>]
```

For an introduction to pro-code development, see Pro-Code Development in Joule Studio.

Setup and Initialization

`jl init <agent>`

Initialize a directory for an AI coding agent. Fetches system skills from the backend and writes MCP configuration files. `<agent>` must be `claude` or `opencode`. See Pro-Code Setup for full details.

```
jl init <agent> [<path>] [--force]
```

Option	Shortcut	Description
`--force`	`-f`	Overwrite existing configuration values and skills.

```
jl init claude
jl init claude my-project
jl init claude -f
jl init opencode
jl init opencode my-project
```

`jl skills init <agent>`

Fetch and install system skills for an AI coding agent without writing MCP configuration files.

```
jl skills init <agent> [<path>] [--force]
```

```
jl skills init claude
jl skills init opencode my-project -f
```

`jl mcp init <agent>`

Write MCP configuration files for an AI coding agent without fetching skills.

- **claude** -- writes `.mcp.json` and `.claude/settings.local.json`.
- **opencode** -- writes `opencode.json`.

```
jl mcp init <agent> [<path>] [--force]
```

```
jl mcp init claude
jl mcp init opencode my-project
```

Authentication

jl login

Log in to a backend. The session is stored locally in encrypted form.

```
jl login ['<url>']
```

Getting the Sign-in URL

Before running `jl login` for the first time, get your personal Sign-in URL from Joule Studio:

1. Open Joule Studio in your browser.
2. Select your user icon in the bottom-left corner and choose **Settings**.
3. Go to **Develop**.
4. Under **CLI Authorization**, copy the **Sign-in URL**.

Pass the URL to `jl login`. Wrap it in single quotes so the shell does not split it at the & characters:

```
jl login '<your Sign-in URL>'
```

A browser window opens for you to authenticate. Once you confirm, the CLI stores the session and you can close the browser tab. After the first successful login, running `jl login` without an argument reuses the stored URL.

jl logout

Log out from a backend. Calls the backend logout endpoint and removes the stored session.

```
jl logout [<url>]
```

Solution

jl solution create

Create `solution.yaml`, an empty `assets/` directory, and a `.build` file containing a stable `solution_id` UUID in the current directory. If `solution.yaml` already exists it is preserved; only missing artifacts are created.

```
jl solution create [--name <name>] [--description <desc>]
```

Option	Shortcut	Description

Option	Shortcut	Description
`--name`	`-n`	Solution name. Optional if `solution.yaml` already exists -- the name is read from `metadata.name` in that case.
`--description`	`-d`	Short description.
`--force`		Overwrite `solution.yaml`, leave `assets/` and `.build` untouched.
`--reset`		Wipe `assets/` and overwrite `solution.yaml`, leave `.build` untouched.

ℹ Note

Note: Names must be DNS-compatible: lowercase letters, digits, and hyphens; 1 to 63 characters; starting and ending with a letter or digit.

ℹ Note

Caution: `--reset` deletes the contents of `assets/`. There is no undo. If you have uncommitted asset files you want to keep, commit them first or back them up.

```
jl solution create --name my-solution --description "Demo solution"
jl solution create   # works if solution.yaml already contains metadata.name
```

jl solution validate

Run a multi-rule validation pipeline against the solution in the current directory.

```
jl solution validate
```

#	Check	Severity
1	`solution.yaml` exists.	error
2	Required fields present (`apiVersion`, `kind`, `metadata.name`, `metadata.version`).	error
3	Conforms to the JSON schema.	error
4	`assets/` directory exists.	error
5	Every `ref` in `assets[]` resolves to an existing file.	error
6	Every subdirectory in `assets/` contains an `asset.yaml`.	warning
7	Every `asset.yaml` has the required envelope fields.	error
8	Every asset passes its plugin-specific validation.	error

The command exits with code `0` if all checks pass and `1` if any error-level check fails.

Fixing validation errors:

Rule	Message	Fix

Rule	Message	Fix
1	`solution.yaml not found at <path>`	Run `jl solution create --name <name>` in the solution root.
2	`apiVersion must be 'solution.sap/v1' / kind must be 'Solution' / metadata.name is missing / metadata.version is missing`	Open `solution.yaml` and correct the named field.
3	`<path> <message>`	Open `solution.yaml` and fix the field at the shown path.
4	`assets/ directory not found`	Run `mkdir assets` in the solution root.
5	`broken refs: <list>`	Check the listed `ref` values in `solution.yaml`. Re-run `jl asset create` or correct the path.
6 (warning)	`missing asset.yaml in: <list>`	Run `jl asset create <type> --name <dir>` for each listed directory, or delete the directory if it is not needed.
7	`<dir>/asset.yaml: type is missing / <issues>`	Open the listed `asset.yaml` and add or correct the named fields.
8	`<dir>: <plugin errors>`	Fix the issue shown. The message is plugin-specific.

jl solution build

Run `jl solution create`, validate the solution, then package it into a zip archive at `build/<name>-<version>.zip`.

```
jl solution build
```

jl solution deploy

Orchestrate a full `create → validate → build → deploy` cycle. If a zip file is provided, skips validate and build and deploys the archive directly (but still runs `create` to ensure `.build` exists).

```
jl solution deploy [--force]
jl solution deploy <zip-file> [--force]
```

```
jl solution deploy
jl solution deploy ./build/my-solution-1.0.0.zip
jl solution deploy --force
```

Option	Short	Description
`--force`		Skip the overwrite confirmation when a solution with this name already exists. Required in non-interactive environments such as CI/CD pipelines.

Troubleshooting:

Error	Cause	Fix

Error	Cause	Fix
`File not found: <path>`	Zip passed as argument does not exist.	Check the path or omit it to let deploy build automatically.
`Deploy aborted`	A solution with this name already exists and you declined the overwrite prompt, or `--force` was not passed in a non-interactive environment.	Re-run with `--force` to overwrite, or rename the solution in `solution.yaml`.
`Deploy failed (403): a solution with this name already exists and is owned by someone else`	Name collision with another owner's solution.	Rename the solution in `solution.yaml` and rebuild with `jl solution build`.
`Deploy failed (401)` or other HTTP errors	Session expired or not logged in.	Run `jl login` and retry.
Deploy succeeds but job fails	Runtime or configuration error on the backend.	Check the logs with `jl solution deploy logs <id>`.

jl solution deploy status

Show the deployment status of a solution. If no ID is given and the terminal is interactive, the command lists available solutions and prompts you to pick one.

```
jl solution deploy status [<id>] [--version <version>] [--json]
```

Option	Description
`--version`	Solution version to inspect (defaults to the first version returned by the backend).
`--json`	Output `{ deploymentInfo, deploymentJobs }` as JSON instead of the default human-readable view.

```
jl solution deploy status my-solution-id
jl solution deploy status my-solution-id --version 1.0.0
```

jl solution deploy logs

Fetch deployment logs for a solution job. If no solution ID or job ID is given and the terminal is interactive, the command prompts you to pick both.

```
jl solution deploy logs [<id>] [--version <version>] [--job <job-id>] [--tail <n>] [--limit <n>] [
```

Option	Description
`--version`	Solution version.
`--job`	Deployment job ID (auto-selected if only one exists; prompted if multiple).
`--tail`	Return only the last N lines.
`--limit`	Maximum number of lines to return.

Option	Description
`--json`	Output the raw logs array as JSON instead of printing each line.

```
jl solution deploy logs my-solution-id
jl solution deploy logs my-solution-id --version 1.0.0 --job abc-123 --tail 100
```

jl solution list

List all solutions on the backend you are logged into.

```
jl solution list [--json]
```

Option	Description
`--json`	Output solutions as JSON instead of the default human-readable list.

Assets

jl asset create

Create a typed asset inside the solution in the current directory. `solution.yaml` must already exist. Run `jl solution create` first if needed.

```
jl asset create <type> --name <name>
```

Option	Shortcut	Description
`--name`	`-n`	Asset name (required).
`--force`		Overwrite if the asset already exists.

Built-in asset types:

Type	Description	Use When
`agent`	Agent with container and health probe configuration.	Building a new agent. Exposes `/.well-known/agent.json`.

jl asset delete

Delete an asset and remove its reference from `solution.yaml`.

```
jl asset delete <name>
```

Argument	Description

Argument	Description
<name>	Name of the asset to delete.

```
jl asset delete my-agent
```

MCP

jl mcp list

List the MCP servers available on the backend.

```
jl mcp list [--json]
```

Option	Description
`--json`	Output servers as JSON instead of the default human-readable list.

jl mcp tools

List the tools exposed by an MCP server, including each tool's parameters. If no server name is given and the terminal is interactive, the command prompts you to pick one.

```
jl mcp tools [<server-name>] [--json]
```

Option	Description
`--json`	Output tools and their input schemas as JSON instead of the default human-readable list.

jl mcp call

Call a tool on a backend MCP server. Parameters are passed as `key=value` pairs, via stdin as JSON, or from a JSON file with `--params`.

```
jl mcp call [<server-name>] [<tool-name>] [key=value ...]
jl mcp call [<server-name>] [<tool-name>] --params <file>
```

```
jl mcp call my-server my-tool key=value
jl mcp call my-server my-tool query="what is api business partners"
echo '{"key":"value"}' | jl mcp call my-server my-tool
jl mcp call my-server my-tool --params params.json
```

`--params <file>` reads the file as JSON and uses it as the tool arguments. Useful for complex or nested parameters. Mutually exclusive with `key=value` pairs.

jl mcp start

Start a local MCP stdio server that proxies all backend MCP servers. Tool names are namespaced as `<server-name>__<tool-name>`.

This command is called automatically by Claude Code and OpenCode based on the configuration written by `jl init claude` or `jl init opencode`. You do not normally need to run it manually.

```
jl mcp start
```

A2A

jl a2a message

Send a message to an agent through the Agent-to-Agent (A2A) protocol and print the response.

```
jl a2a message <message> [--solution <id>] [--asset <name>] [--continue]
```

Option	Shortcut	Description
`--solution`	`-s`	Solution ID. Defaults to `solution_id` from `.build` if present.
`--asset`	`-a`	Asset name of the target agent. Defaults to the single `agent`-type asset in `assets/` if unambiguous.
`--continue`	`-c`	Reuse the last conversation context ID (stored in `.build` as `a2a_context_id`). Starts a new context if omitted.

Each successful call saves the context ID to `.build` so `--continue` can resume the conversation in a subsequent call.

```
# Explicit options
jl a2a message "What is the status?" --solution my-solution-id --asset my-agent

# Defaults resolved from .build and assets/
jl a2a message "What is the status?"

# Continue the previous conversation
jl a2a message "Tell me more" --continue
```

Utilities

jl version

Print the installed CLI version.

```
jl version
```

jl info

Show the CLI version, Node runtime, platform, active backend endpoint, and API version.

```
jl info
jl info --json
```

Option	Description
`--json`	Output as JSON.

Shortcuts

The CLI accepts the following shortcuts to keep commands concise.

Shortcut	Expands to
`s`	`solution`
`a`	`asset`
`v`	`version`

```
jl s create --name my-sol        # jl solution create --name my-sol
jl a create agent --name x       # jl asset create agent --name x
```

Related Information

- Pro-Code Development in Joule Studio
- Pro-Code Setup
- Pro-Code Workflow
- Solution Structure

The General Development Flow

This section describes the overall workflow in Joule Studio. Each step links to a dedicated page with detailed guidance.

You'll learn about the details of the AI-driven development flow in Joule Studio from intent to solution in a few easy steps. Each step builds on the previous one and requires explicit approval, ensuring human validation at every stage.

You can perform each step in the browser-based Joule Studio UI or from the terminal using the `jl` CLI, optionally with an AI coding agent. For the CLI mapping, see Pro-Code Workflow.

Workflow Steps

Step	Description	Output
Intent	Describe your challenge; Joule Studio clarifies your intent and maps it to SAP architecture.	Intent document
Requirements	AI generates a business requirements document.	Product Requirements Document (PRD)
Specification	AI creates a technical design and task list.	Technical specification
Solution	AI generates production-ready code.	Deployable solution
Testing	Validate your solution in the sandbox before production.	Verified functionality
Deployment	Deploy to the managed runtime service.	Running solution with monitoring

After Deployment

Once your solution is deployed to the development environment, the following topic covers what comes next.

- GitHub Setup: Set up GitHub and GitHub Actions to enable production deployment.

Intent

The Intent step shapes the entire solution that Joule Studio builds for you. A well-defined intent leads to more accurate requirements, better API selections, and a solution that closely matches your actual business needs.

The Intent step consists of AI-driven discovery as well as clarifying questions (disambiguation).

AI-Driven Discovery

Joule Studio initiates an AI-driven discovery process:

1. Reference Business Architecture Mapping via SAP Knowledge Graph: Maps the business challenge to SAP's process hierarchy.
2. SAP LeanIX Query: Discovers your SAP landscape (business capabilities, applications, interfaces, IT components, and ongoing initiatives).
3. Fit-Gap Analysis and API Discovery via SAP Knowledge Graph: Identifies relevant SAP APIs and evaluates standard assets versus custom development needs.
4. SAP Domain Models Consultation: Applies SAP-opinionated architecture patterns and best practices.

Clarifying Questions (Disambiguation)

Joule Studio captures and validates your business problem through clarifying questions. These questions help define the solution scope and validate the AI's assumptions about your business challenge.

Artifacts at this Stage

- intent.md: The single output artifact of this step. It consolidates the RBA mapping, fit-gap analysis, and solution recommendation into one document.

User Actions and Responsibilities

- Respond to clarifying questions with specific business context.
- Include quantifiable success metrics for the intended solution where applicable.
- Specify known system constraints or integration requirements.
- Confirm or correct landscape assumptions.
- Approve the analysis to proceed to PRD generation.

Next Steps

- Intent Refinement with AI-Driven Discovery: Refine your intent statement to Joule Studio.

Intent Refinement with AI-Driven Discovery

Intent refinement begins automatically when you provide your intent statement to Joule Studio.

During this process, Joule Studio queries multiple SAP intelligence systems to understand your landscape, map your challenge to SAP architecture, and identify the optimal solution approach. Specifically, it:

- Maps your challenge to SAP's Reference Business Architecture via SAP Knowledge Graph
- Queries your SAP landscape using SAP LeanIX
- Discovers relevant APIs and performs a fit-gap analysis via SAP Knowledge Graph
- Checks the SAP Domain Models for best practices

This process allows Joule Studio to turn your business challenges and ideas into a structured foundation for solution development.

AI-Driven Discovery

Step 1: Reference Business Architecture Mapping with SAP Knowledge Graph

Joule Studio maps the business challenge to SAP's Reference Business Architecture using SAP Knowledge Graph, identifying the relevant process hierarchy.

```
End-to-End Process (e.g., Source to Pay)
└── Phase (e.g., Procurement)
    └── Sub-Process (e.g., Purchase Order Processing)
        ├── Business Activity (e.g., Create Purchase Order)
        └── Business Activity (e.g., Approve Purchase Order)
```

Step 2: Enterprise Landscape Discovery with SAP LeanIX

SAP LeanIX grounding ensures that solution recommendations match the actual implemented landscape rather than assuming a generic SAP environment.

Joule Studio queries SAP LeanIX to discover:

Information	Importance for Solution Generation
Business Capabilities	Identifies what the organization can already do in the challenge domain
Applications	Implemented SAP systems, for example SAP S/4HANA or SuccessFactors, and on-premise or cloud
Interfaces	Integration points between SAP applications
IT Components	Technical dependencies and infrastructure
Ongoing initiatives	Any ongoing projects or initiatives that may impact or inform the solution

Step 3: Fit-Gap Analysis and API Discovery with SAP Knowledge Graph

In this step, Joule Studio evaluates which standard SAP assets fit the business requirements, identifying what standard components can be used (**fit**) versus what requires custom development in Joule Studio (**gap**).

The goal of fit-gap analysis is to ensure that existing capabilities in SAP's standard portfolio are leveraged and not redeveloped, while clearly identifying where custom development adds value.

When custom development is needed, SAP Knowledge Graph also discovers:

- **APIs**: Available endpoints with specifications
- **Data assets**: Existing data sources from SAP systems
- **Business events**: Event schemas for event-driven architectures

Example

A solution that checks overdue invoices, calculates the missed discount impact, and sends personalized collection emails, requires fit-gap analysis. Joule Studio checks the SAP Knowledge Graph to determine whether any of these capabilities are already available.

The **API ORD ID** is the unique identifier of an SAP API (such as an OData service) registered in the SAP Knowledge Graph. The **MCP Server ORD ID** identifies a pre-built MCP server that wraps that API, allowing an agent to call it directly as a tool. If an MCP server exists for a given API, Joule Studio records its ORD ID and uses it when building the solution. If only an API ORD ID is found but no MCP server exists, Joule Studio generates a new MCP server asset as part of the solution. A dash (—) means no standard asset was found for that requirement.

Requirement	Standard Asset	API ORD ID	MCP Server ORD ID	Gap	Notes
View overdue invoices	SAP S/4HANA Supplier Invoice API	—	—	No	Standard API covers requirement
Calculate missed discount impact	None	—	—	Yes	Custom calculation logic required
Send personalized collection emails	None	—	—	Yes	AI-generated content required

Step 4: Best-practice validation with SAP Domain Models

After the fit-gap analysis identifies a solution direction, Joule Studio validates the recommendation using SAP Domain Models.

SAP Domain Models comprise the intelligence layer for enterprise AI. They combine SAP's purpose-built domain models with third-party LLMs. SAP models are trained on application code, process knowledge, and industry expertise. SAP Domain Models enable accurate validation of solution recommendations against SAP best practices.

The validation applies SAP's architecture principles:

Principle	Application

Principle	Application
Business before Technology	Solutions derived from requirements, not technology preferences
Cloud First	Prefer cloud services for new capabilities
Run Simple	Choose the simplest architecture that meets requirements
Extensibility	Standard features first; custom only for differentiation
Data as Asset	Treat data quality as competitive advantage; single source of truth
Composability	Modular services around a lean core
Control Technical Debt	Use maintained, supported technology stacks

Discovery Outputs

The discovery pipeline produces an intent document containing:

Section	Content
Business Challenge	The original requirement, clarified and validated
Key Milestones	Checkpoints marking meaningful progress in the solution
Business Architecture	Reference Business Architecture mapping with process hierarchy
Fit-Gap Analysis	Requirements mapped to standard assets with gap identification
Recommendation	Proposed solution with executive summary, affected roles, and solution category

The recommendation also includes an **intent fit** percentage, indicating how closely the proposed solution matches the analyzed business challenge.

Next Steps

The intent document flows directly into the Requirements step, where Joule Studio generates a Product Requirements Document (PRD).

The discovery insights (landscape context, Reference Business Architecture mapping, fit-gap analysis, and validated recommendations) inform every subsequent step of development.

Requirements

A clear set of requirements ensures that the generated solution addresses your actual business needs. This is your opportunity to verify scope, personas, and success criteria before Joule Studio begins technical design.

The Requirements step produces a Product Requirements Document (PRD) based on validated intent analysis.

What the Requirements Page Shows

The **Requirements** page displays the generated PRD as a structured document with the following:

- **Header metadata**: Title, Date, Owner, and Solution Category (for example, ***AI Agent***).
- **Expandable sections**: Each PRD section is collapsible so you can focus on the part you are reviewing.
- **Click to edit**: A control at the top of the page lets you open the PRD for editing, so you can refine sections directly.

To advance from the Idea Board to the Requirements step, select **Create Requirement** on the Idea Board card, or type **Build requirement for this project** in the Joule chat.

Process Overview

The AI synthesizes intent analysis outputs into a structured PRD containing:

- Product Purpose and Value Proposition: Business problem, expected value, and prioritized objectives
- User Personas: Target users with their goals, tasks, and workflows
- Product Principles: Design principles guiding decision-making
- Business Context: Current state, strategic alignment, and success criteria
- Goals and Non-Goals: Scope definition with explicit inclusions and exclusions
- Requirements: Each requirement (R01, R02, ...) includes a problem to solve, a user story, acceptance criteria, the objective it maps to, and a priority rank. Requirements are grouped as must-have, high-want, and nice-to-have.
- Non-Functional Requirements: Performance, reliability, cost, and explainability constraints
- Solution Architecture: Architecture overview, key components, integration points, and subsections covering agent extensibility, instrumentation, and the agent's automation and behavior.
- Milestones (M1, M2, ...): Each milestone includes a description, the condition that signals when it is achieved, and structured log statements for both successful achievement and miss scenarios. Milestones support production observability after deployment.
- Risks, Assumptions, and Dependencies: Known risks, unverified assumptions, and external dependencies

Note: In fast-track mode, some sections are omitted to accelerate development.

Artifacts at this Stage

- product-requirements-document.md: The single output artifact of this step. Contains the complete requirements specification.

User Actions and Responsibilities

- Review all PRD sections for accuracy and completeness.
- Confirm that user personas reflect actual organizational roles.
- Verify that success criteria are measurable and testable.
- Use the Joule chat to request changes if a section needs refinement.
- Move on to the specification step when the PRD is ready.

Next Steps

- Specification: Translate approved requirements into an implementation blueprint.

Specification

The specification defines exactly how your solution will be built. Reviewing it helps you catch architectural mismatches early, before code generation begins.

The Specification step translates requirements into an implementation blueprint. While the PRD is the preferred input, the specification can also be generated directly from the intent analysis.

What the Specification Page Shows

The **Specification** page displays a per-asset specification, with one panel per asset in your solution. For each asset, the page shows a list of **expandable steps** that cover the work needed to implement that asset. The steps and their content vary by solution type.

A step number next to each asset (for example, ***equipment-replacement-agent · 10 steps***) indicates how many implementation steps the AI generated.

To advance from the Requirements step to the Specification step, type **Build specification for this project** in the Joule chat.

Process Overview

The AI generates a per-asset specification that defines, for each asset:

- Setup tasks: Project initialization, dependencies, and basic configuration
- API and tool integration: How the asset connects to SAP APIs and MCP tools. Tool schemas are captured as JSON files.
- Implementation tasks per requirement: A step-by-step plan for each functional requirement (R01, R02, ...) from the PRD
- Instrumentation: Structured log statements and OpenTelemetry spans for the milestones defined in the PRD
- Configuration files: `solution.yaml` and `asset.yaml` updates needed for the asset
- Testing tasks: Unit tests per tool, integration tests, and a coverage target
- Validation checklist: A final review of all artifacts before code generation

Artifacts at this Stage

The specification is generated as a folder structure containing multiple files:

- A master specification (`specification.md`) covering the solution as a whole
- Guidelines files that the AI follows during code generation: `guidelines.md` and `guidelines-<type>.md` (such as `guidelines-agent.md`).
- A per-asset folder for each asset, containing:
    - The asset's own `specification.md` with a step-by-step task list
    - An `mcp-specs/` directory with JSON schemas for each MCP tool the asset uses
    - An `api-specs/` directory (optional): Downloaded OData/REST API spec files used during code generation

User Actions and Responsibilities

- Review architecture decisions and the integration approach for each asset.
- Verify that the proposed architecture aligns with your organizational standards.
- Use the Joule chat to add implementation constraints or change task priorities.
- Move on to the solution step when the specification is ready.

Next Steps

- Solution: Produce deployment-ready code based on the approved specification.

Solution

This is where your requirements become a working solution. Understanding what Joule Studio generates helps you review the output effectively and request targeted changes.

The Solution step produces deployment-ready code based on the approved specification.

What the Solution Page Shows

The **Solution** page is where you review, run, and refine the generated solution. Each asset in the solution has its own page, accessed from the breadcrumb at the top.

The Solution page offers three modes, available as buttons at the top:

- **Try**: A chat box where you can send prompts to the running agent and inspect its responses. Below the chat, a **Traces** panel shows the agent's tool calls and reasoning steps; switch to the **Logs** sub-tab to inspect execution details.
- **Design**: A configuration editor where you can review and adjust the agent's settings (the system prompt, the LLM configuration, and the MCP servers it uses) without editing code.
- **Code**: A file browser with a search box and a tree view of the generated files. Select a file to view its contents.

A **Deploy** button is available at the top right of the Solution page so you can deploy the asset directly from this view, without navigating to the Deployment page.

Process Overview

The AI generates complete, ready-to-deploy solution artifacts:

- Agents: Python implementation with MCP tool definitions, agent SDK integration, and system prompt configuration

All generated solutions include:

- Compliance with SAP Product Standards (security, scalability, accessibility)
- Error handling and retry logic
- Audit logging for monitoring and compliance
- Secure, pre-configured connections to your SAP systems

Artifacts at this Stage

- Generated code: Complete implementation in the code editor panel
- Project structure: Solution organization following SAP conventions
- Configuration files: Deployment descriptors and environment templates

User Actions and Responsibilities

- Try out the solution and check the results.
- Review the generated code for alignment with organizational coding standards.
- Verify that API calls match the specification requirements.
- Confirm that error handling addresses the expected failure modes.
- Modify the code using the Joule chat, the **Design** mode, or the **Code** mode, depending on what you want to change.
- Request regeneration of specific components as needed.
- Proceed to testing when the implementation is satisfactory.

Next Steps

- Testing: Review the automated test report and validate the solution before production.

Testing

Testing ensures your solution works as expected before it reaches production users. Catching issues here avoids disruptions to your business processes later.

The Testing step validates the solution before production deployment. Joule Studio runs automated tests against the generated code and reports the results on the Testing page; you can also interact with the running agent on the Solution page to confirm it behaves as expected.

What the Testing Page Shows

The **Testing** page presents a report dashboard with the results of the automated tests Joule Studio runs against your solution:

- **Total Tests**: number of test cases generated for the solution
- **Passed** / **Failed**: how many tests succeeded or failed in the most recent run
- **Coverage**: percentage of solution code covered by tests
- **Score**: overall test quality score for the solution
- **Validation by Artifact**: per-asset test results, with a percentage score for each asset

If no tests have been run yet, the page shows "No test report found."

Trying Your Agent Interactively

Interactive testing (chatting with your agent before it goes live) happens on the **Solution** page in **Try** mode, not on the Testing page. The Solution page provides:

- A chat box where you can send prompts to the running agent.
- A **Traces** panel that shows the agent's tool calls and reasoning steps.
- A **Logs** panel for inspecting execution details.

Use the Solution page's Try mode for functional testing and exploratory interaction. Use the Testing page to review the automated test report.

Generated Tests

Joule Studio generates tests during solution generation for most solution types. These tests verify that data connections, business rules, and response formatting all work as expected.

Test Coverage by Solution Type

Solution Type	Test Coverage

Solution Type	Test Coverage
Agents	Tool invocations, response formatting, error handling

Test Execution

Tests run as part of the solution validation process. Results appear in the **Testing** panel with pass/fail status and execution logs.

AI-Powered Evaluation

Joule Studio uses an AI-powered evaluation step to verify that the generated solution aligns with the original intent.

The evaluation compares:

- PRD requirements against implemented functionality
- Technical specifications against generated code
- Expected outcomes against actual behavior

Evaluation Metrics

Metric	Description
Intent Alignment	Measures how well the solution addresses the original business challenge.
Requirements Coverage	Verifies that all PRD requirements are implemented.
Specification Adherence	Confirms that the generated solution matches the approved design.

User Actions and Responsibilities

- Review the generated tests in the **Testing** panel.
- Check for errors or unexpected behavior in the test report.
- Try your agent interactively in **Solution** → **Try** to validate behavior on real prompts.
- Use the Joule chat to ask the AI coding agent to fix failing tests, add tests, or adjust behavior.
- Approve for production deployment when testing is complete.

Next Steps

- Deployment: Provision the solution to the managed runtime environment.

Deployment

Overview

Deployment is the final step that makes your solution available to users. Understanding this process helps you monitor progress and verify that everything is running correctly.

The Deployment step provisions your solution to the managed runtime service. SAP handles infrastructure, scaling, and operations. The deployment target and process vary by solution type.

Solution Type	Deployment Target
Agent	Kyma, registered with Joule

Deployment Target

All Joule Studio solutions deploy to the Joule Studio runtime, a managed runtime service.

Managed Runtime Capabilities

Capability	Description
Managed Infrastructure	SAP handles scaling, updates, and resource management
Security	Enterprise-grade authentication and data protection
Observability	Logging, monitoring, and alerting included
Connectivity	Secure connections to SAP and third-party systems

ℹ Note

Note: When a user interacts with a deployed agent, the agent accesses SAP data using that user's own credentials and authorizations (principal propagation). The agent cannot retrieve data that the user is not already permitted to see.

Deployment Process

When you approve deployment, Joule Studio executes the following steps:

1. Validate: Checks the solution for errors before building
2. Build: Packages the solution with all dependencies and configuration
3. Deploy: Provisions and starts the runtime environment
4. Check status: Confirms the deployment completed successfully

The process is automated end-to-end. For agent-type solutions, an additional step registers the agent with the Agent Gateway after deployment.

How to Deploy

The steps below apply to the **development environment**. Deploying to the productive environment uses a separate GitHub-based process, where a developer pushes the solution to a GitHub repository and an administrator triggers a GitHub Actions workflow as the approval gate. See GitHub Setup for details.

You can start a development deployment from any of these places:

- The **Deploy** button at the top right of the **Solution** or **Testing** page.
- The **Deployment** step in the progress bar, after at least one test has run.
- The Joule chat, by typing **deploy the solution**.

When you select Deploy, a small popover appears asking you to choose a deployment environment. Select **Deploy** in the popover to start the deployment.

While deployment is running, the page header shows a **Deploying...** badge, and the chat panel monitors progress in the background. When deployment completes successfully, the header switches to a **Deployed** badge and the chat displays a deployment summary with:

- The deployed agent's URL
- The asset name and version
- The deployment space
- The deployment timestamp

What the Deployment Page Shows

The **Deployment** page lists every deployment you have started for the current solution, most recent first. Each entry shows:

- A deploy number (for example, ***Deploy No. 1***)
- When the deployment was created and by whom
- The target environment (for example, ***Development***)
- The current status (***Deploying***, ***Deployed***, or ***Failed***)

Use the status filter at the top right to narrow the list to deployments in a specific state.

Deployment Artifacts

After successful deployment, the following artifacts are available:

Artifact	Description
Deployment Log	Real-time progress and status

Artifact	Description
Runtime Endpoints	URLs for accessing the deployed solution
Monitoring Dashboard	Health, performance, and usage metrics
Solution Overview	Summary of deployed assets and status

Runtime Capabilities

Deployed solutions include the following runtime capabilities:

- Horizontal scaling: The platform uses multiple CF instances to handle load
- Security: Enterprise authentication and secure credential management
- Monitoring: Health checks, performance metrics, and configurable alerts
- Logging: Centralized logs for debugging and auditing

User Actions and Responsibilities

- Monitor deployment progress in the Deployment page or in the chat panel.
- Verify that the page header switches to **Deployed** and that the deployment summary lists the expected agent URL.
- Access the deployed solution using the URL in the deployment summary.
- Retry deployment from the same page if errors occur.

Post-Deployment Operations

After deployment, you can manage the solution from the Solution Overview in Joule Studio:

- Monitor: View health, performance, and usage metrics
- Operate: Start, stop, or redeploy the solution
- Iterate: Modify the solution and redeploy

Downloading Your Solution

After deployment, you can download the complete solution files as a ZIP archive at any time. Select the **More options** button (the three-dot icon) in the top-right corner of any page within the solution, then choose **Download ZIP**. The archive contains all solution files, including `solution.yaml` and the solution files generated during the build. You can use this to share the solution with a colleague or keep a local copy.

Monitoring Deployed Solutions

The solution overview includes a **Monitoring** tab that shows runtime metrics for the deployed solution over a configurable time range (the default is ***Last 7 days***):

- **Failed Agent Sessions**: Sessions that failed while running
- **Workflow Errors**: Workflow executions that ended with errors
- **Completed Workflows**: Successfully completed workflows
- **Completed Agent Sessions**: Agent sessions that completed successfully

A per-asset table lists incidents for each artifact in the solution, with columns for the number of errors and the last incident time. Use this view to monitor production health and to investigate failed runs.

Deploying to Production

Development deployment (described above) makes your agent available in the development environment only. To promote an agent to the productive environment, you use a GitHub-based process:

1. In Joule Studio, connect your solution to a GitHub repository and push the solution files to GitHub directly from Joule Studio.
2. Notify your administrator that the solution is ready for production. There is no automated notification; this step is manual (email, Slack, or in person).
3. The administrator goes to the GitHub repository, opens the Actions tab, and manually triggers the deployment workflow. This manual trigger is the deliberate approval gate for production.
4. Once the workflow completes, the agent is live in the productive environment. Deployment status is visible in GitHub Actions, and the administrator can also view the solution and its observability data in the productive Joule Studio.

For the one-time setup required before the first production deployment, see GitHub Setup.

Related Topics

- Testing
- Solution Types
- Key Concepts
- GitHub Setup: Set up GitHub and GitHub Actions to deploy to the productive environment.
- Assign Agent Access: After deploying to production, control which end users can invoke the agent through Joule.

Assign Agent Access

Overview

After deploying an agent to the productive environment, you control which end users can invoke it through Joule. This is separate from the Joule Studio roles described in Role Management:

- **Joule Studio roles** (`JouleStudioDeveloper`, `JouleStudioAdministrator`) control who can build, deploy, and manage solutions inside Joule Studio.
- **Agent access policies** control which end users can invoke a specific deployed agent through Joule.

Both are managed in SAP Cloud Identity Services, but they are distinct assignments.

> **Agent access policies for custom agents are not yet available**
> Until this feature is enabled, any user with access to Joule can invoke a deployed agent. The procedure below applies once agent access management becomes available (expected end of July 2026).

How Agent Authorization Works

When a user invokes a deployed agent through Joule, the agent accesses SAP data using that user's own credentials and authorizations (principal propagation). The agent cannot retrieve data the user is not already permitted to see in the underlying system. Granting access to an agent does not expand a user's data permissions — it only controls whether they can invoke the agent at all.

Procedure

Prerequisites

- The agent has been deployed to the productive environment. For the deployment procedure, refer to Deployment.
- You have administrator access to the productive SAP Cloud Identity Services tenant.

Step 1: Open the Agent in SAP Cloud Identity

1. Sign in to the SAP Cloud Identity Services administration console for the productive environment. The URL has the form `https://<your-sci-tenant>.accounts.ondemand.com/admin`.
2. Navigate to the **Agents** page.
3. Select the agent you want to configure.

Step 2: Assign or Remove Users

1. Choose **Manage Access**.

2. In the dialog, select the users you want to grant access to, or deselect users to remove access.

3. Choose **Save**.

Users you have assigned can now invoke the agent through Joule. Users without an assignment will not be able to call the agent.

Next Steps

- Continuous Improvement: Update or offboard a deployed agent.
- Role Management: Assign Joule Studio Developer and Administrator roles to builders and admins.
- Deployment: Deploy your agent to the development and productive environments.

Solution Types

Overview

Joule Studio generates agents that connect to your SAP systems, answer questions, and take action on your behalf.

> **Note**
>
> **Note:** Joule Studio may also recommend using an existing SAP product that already covers your requirements. In that case, no custom solution is generated - Joule Studio guides you to the standard SAP capability instead.

Agents

What is an Agent?

An Agent is a conversational AI assistant that connects to SAP systems to answer questions and take actions. Agents communicate with SAP backends through a secure gateway and integrate with Joule for natural-language interaction.

Use Cases

- Natural language queries over SAP data
- Multi-step data retrieval and analysis
- Autonomous recommendations based on business logic
- Intelligent assistants embedded in workflows

Runtime Behavior

- Deploys to a managed runtime service
- Registers with Joule for conversational access
- Connects to SAP systems via Agent Gateway
- Supports extensibility

Example

Missed Discount Analysis Agent

Prompt: ***"Create a new agent that does a missed discount analysis, taking the unmatched invoices into account and analyzing how much discount I'm losing because I cannot match the invoices and issue the payment on time."***

The agent connects to SAP S/4HANA, retrieves unmatched invoice data, analyzes payment terms and discount windows, and returns a financial impact summary.

Solution Type Selection

Joule Studio automatically recommends a solution type based on your intent. The AI-driven discovery process evaluates your requirements and proposes the best fit.

Business Need	Recommended Solution
Natural language queries, autonomous analysis	Agent

Navigating Your Solutions

Your solutions are listed in the left panel of Joule Studio. Selecting a solution opens an overview in the center area, showing its description, creation details, and artifacts. To open the solution and work with its content, select **Open** in the top-right corner.

Agents

What Are Agents?

Agents are AI-powered assistants built in Joule Studio that understand natural language, access SAP data, and execute tasks. They function as intelligent digital assistants that answer questions, perform operations, and provide insights through conversation.

An agent:

- **Understands natural language**: You can ask questions or give commands in plain language.
- **Accesses SAP systems**: Connects to SAP S/4HANA and other systems through MCP (Model Context Protocol) tools via Agent Gateway.
- **Uses tools**: Performs specific actions (query data, analyze trends, generate insights).
- **Makes decisions**: Determines which APIs to call based on context.
- **Communicates with other agents**: Discovers and invokes other agents via the A2A (Agent-to-Agent) protocol.

To create an effective agent, you should define a clear intent statement by describing the specific analysis or tasks that the agent should perform, the exact SAP data it requires for access, and the business challenge you are aiming to solve. Additionally, you must ensure the necessary system access is in place, which includes an SAP S/4HANA system configured with a BTP destination, functional Agent Gateway connectivity, and the appropriate authorizations for any data the agent will need to retrieve or process.

How Agents Work

main.py: Defines the agent's identity and capabilities:

- Agent Card (name, description, version, URL)
- Agent Skills (what the agent can do)
- A2A server configuration

agent.py: Contains the agent logic:

- System prompt defining agent behavior
- LangGraph state machine for conversation flow
- LLM integration via LiteLLM

Agent Capabilities

Skills

A **skill** is a specific capability an agent can perform, for example, retrieving invoice data, calculating a financial impact, or summarizing supplier performance. Skills define the scope of what the agent can do.

SAP System Connectivity

Agents access SAP systems through a secure gateway that serves as the single, governed connection point between your agents and SAP backends. This means:

- No direct database access or uncontrolled connections
- Connectivity is established once during deployment and reused consistently
- Access is governed by your organization's existing authorization model

Discovery by Joule

Once deployed, agents are automatically discoverable through Joule. When users ask questions that match the agent's area of responsibility, Joule routes the request to the agent. Users don't need to know that a custom agent exists or how to find it.

Example: Missed Discount Analysis Agent

Business Problem: Unmatched invoices go unnoticed, leading to:

- Lack of visibility into missed payments
- Unknown financial impact from lost early payment discounts

Prompt:

```
Create a new agent that does a missed discount analysis,
taking the unmatched invoices into account and analyzing how much discount I'm losing
because I cannot match the invoices and issue the payment on time.
```

What the agent does:

- Reads and analyzes unmatched invoices from SAP S/4HANA
- Calculates lost early payment discounts
- Quantifies the financial impact due to delayed or missed payments

System dependencies: SAP S/4HANA, Agent Gateway

Key Takeaways

1. **Agents are conversational**: Users interact in plain language through Joule; no separate tool or training required
2. **Joule handles routing**: The right agent is invoked automatically based on what the user asks
3. **Secure by design**: All SAP system access goes through a governed gateway with no uncontrolled data exposure
4. **Deployed and managed by SAP**: Runs on SAP's cloud platform with built-in monitoring and scaling
5. **Built end-to-end in Joule Studio**: From intent to live agent, with no manual coding required

Related Information

- Deployment: Understand how solutions go live.

Continuous Improvement

The following capabilities and functionalities provided by SAP as part of SAP Joule Studio, including code generation, system context , recommendations, and related generative AI capabilities, may be continuously improved by SAP on behalf of the customer.

SAP may use Customer Data, in accordance with the applicable contractual terms, in order to support such improvements:

SAP Joule Studio enables customers to create, configure, adapt, and operate agentic solutions, automations, applications, and related artifacts. As part of SAP's provisioning and continuous improvement of these capabilities, SAP may process Customer Data to improve the quality, reliability, accuracy, relevance, performance, and usability of the underlying services and models. Such improvements may include reducing errors, improving interpretation of business context, enhancing generated outputs, increasing execution reliability, improving response quality, optimizing processing time, and supporting better user guidance.

Categories of Customer Data that may be processed for these purposes may include business process data, user-provided instructions, configuration data, metadata, execution logs, system interaction data, user feedback, and other data submitted, generated, or processed through SAP Joule Studio in the context of the customer's use of the service.